Platzhalter

Lo
go

Term paper

Sample title for a report

Author:	First name Last name
Matriculation No.:	xxxxxxx
Course of Studies:	Business Intelligence and Data Analytics
First examiner:	Prof. Dr. Elmar Wings
Submission date:	October 12, 2025

University of Applied Sciences Emden/Leer · Faculty of Technology · Mechanical Engineering Department
Constantiaplatz 4 · 26723 Emden · http://www.hs-emden-leer.de

Contents

Contents 3

List of Figures 9

List of Listings 11

1. Introduction 15
1.1. Problem Description . . . 15
1.1.1. Historical Context: Evolution of ANPR . . . 16
1.1.2. Why YOLO-based Edge Systems Are a Breakthrough . . . 16
1.1.3. Comparison with Traditional ANPR Cameras . . . 17
1.2. Literature: Motivation and Relevance . . . 18
1.2.1. Case Studies: Airports and Deployments in Germany . . . 18
1.2.2. Cost-Benefit Analysis . . . 19
1.3. High-Level System Objectives . . . 19
1.3.1. Objective 1: Detection Pipeline . . . 19
1.3.2. Objective 2: OCR Pipeline . . . 20
1.3.3. Objective 3: Local SQL Logging and Billing . . . 20
1.3.4. Objective 4: Privacy and Legal Compliance . . . 21
1.3.5. Objective 5: Field Deployment Guidelines . . . 21
1.4. YOLOv8 Approach . . . 22
1.4.1. Why Two Models? . . . 22
1.4.2. Comparison with Other Detectors . . . 22
1.4.3. Advantages of YOLOv8 for Edge ANPR . . . 22
1.4.4. Edge Deployment Considerations . . . 23
1.5. Target Hardware: NVIDIA Jetson Nano . . . 23
1.5.1. Hardware Specifications . . . 23
1.5.2. Justification for Choice . . . 23
1.6. German License Plate Types . . . 24
1.7. Challenges and Solutions . . . 25
1.7.1. Lighting Variations . . . 25
1.7.2. Weather Conditions (Rain, Dust, Snow) . . . 25
1.7.3. Angle and Perspective Distortion . . . 25
1.7.4. Night-time Headlight Glare . . . 26
1.7.5. Dirty, Occluded, or Damaged Plates . . . 26
1.7.6. Computation and Real-Time Constraints on Edge Devices . . . 26
1.7.7. False Positives Due to Background Text . . . 26
1.7.8. Privacy and GDPR Compliance . . . 27
1.7.9. Tampered or Fake Plates . . . 27
1.7.10. Environmental and Operational Edge Cases . . . 27
1.8. System Architecture . . . 29
1.8.1. Camera Setup . . . 29
1.8.2. Inference Pipeline . . . 29
1.8.3. Database Schema . . . 29
1.8.4. Billing Logic . . . 29
1.9. OCR Methods . . . 33

1.10. Example Scenarios . . . 33
1.11. Legal and Privacy Considerations . . . 33
1.12. Limitations and Future Work . . . 34
1.13. Summary . . . 34

I. Domain Knowledge - Application **35**

2. Application Domain Knowledge **37**
2.1. Application Domain Knowledge . . . 37
2.1.1. Domain Knowledge — Application Domain . . . 37
2.1.2. Application Problem . . . 37
2.1.3. Data Acquisition . . . 37
2.1.4. Data Quantity . . . 38
2.1.5. Data Quality . . . 39
2.1.6. Data Relevance . . . 39
2.1.7. Outliers . . . 39
2.1.8. Anomalies . . . 39

II. Domain Knowledge - Hardware **41**

3. Jetson Nano **43**
3.1. Description . . . 43
3.2. Applications of the Jetson Nano . . . 44
3.3. Construction . . . 46
3.4. Interfaces . . . 49
3.4.1. J1 - J19 Connectors . . . 49
3.4.2. Pin Configuration . . . 50
3.5. Data Quality . . . 51
3.6. Data Quantity . . . 51
3.7. Specification of Jetson Nano . . . 51
3.8. Constraints . . . 51
3.9. Dimension of Jetson Nano . . . 51
3.10. Simple Code for Jetson Nano Reference Carrier Board . . . 52

4. Sensor XY **53**
4.1. General . . . 53
4.2. Specific Sensor/Actor . . . 53
4.3. Specification . . . 53
4.4. Library . . . 53
4.4.1. Description . . . 53
4.4.2. Installation . . . 53
4.4.3. Functions . . . 53
4.5. Example . . . 53
4.5.1. Manual . . . 54
4.5.2. Inside the Example . . . 54
4.5.3. Code . . . 54
4.5.4. Files . . . 54
4.6. Calibration . . . 54
4.7. Simple Code . . . 54
4.8. Simple Application . . . 54

4.9. Tests . . . 54
4.9.1. Simple Function Test . . . 54
4.9.2. Test all Functions . . . 54
4.10. Further Readings . . . 54

5. Actor XY **55**
5.1. General . . . 55
5.2. Specific Sensor/Actor . . . 55
5.3. Specification . . . 55
5.4. Library . . . 55
5.4.1. Description . . . 55
5.4.2. Installation . . . 55
5.4.3. Functions . . . 55
5.5. Example . . . 55
5.5.1. Manual . . . 56
5.5.2. Inside the Example . . . 56
5.5.3. Code . . . 56
5.5.4. Files . . . 56
5.6. Calibration . . . 56
5.7. Simple Code . . . 56
5.8. Simple Application . . . 56
5.9. Tests . . . 56
5.9.1. Simple Function Test . . . 56
5.9.2. Test all Functions . . . 56
5.10. Further Readings . . . 56

III. Domain Knowledge - Machine Learning/Algorithms **57**

6. Algorithm XY **59**

IV. Domain Knowledge - Tools **61**

7. Python Tool XY **63**

8. Hardware Tool XY **65**

V. Methodology **67**

9. KDD XY **69**

VI. Developmenmt **71**

10. Flow Chart Development XY **73**

11. Database XY **75**

12. Data Selection XY **77**

13. Data Transformation XY **79**

14. Data Mining XY 81

VII. Development to Deployment 83

15. Machine Learning Pipeline XY 85

16. Database XY 87

VIII Deployment 89

17. Flow Chart Deployment XY 91

IX. Monitoring 93

18. Monitoring XY 95

X. Verification - Evaluation - Conclusion 97

19. Evaluation/Verification XY 99

20. To DoX Y 101

21. Conclusion XY 103

XI. Appendix 105

22. Bill of Material 107

23. SBOM 109

24. Package Example 111
24.1. Introduction . . . 111
24.2. Description . . . 111
24.3. Installation . . . 111
24.4. Example - Manual . . . 111
24.5. Example . . . 111
24.6. Example - Code . . . 111
24.7. Example - Files . . . 111
24.8. Further Reading . . . 111

XII. Hints - Examples for LaTeX - Read and Use 113

25. Hints 115
25.1. Allgemeine mathematische Beschreibung Bézier-Kurve . . . 116

26. Verrundung mit einem Kreisbogen 119
26.1. Gleichungen . . . 119
26.2. Bewertung . . . 121
26.3. Examples . . . 122

27. Maple files **125**
27.1. Geometries with Maple . . . 125
27.2. Geometry elements used . . . 125
27.3. Building the data structure . . . 126
27.4. Structure of a module . . . 126
27.4.1. General structure of a module . . . 128
27.4.2. Saving a module . . . 129
27.4.3. Verwendung eines Moduls . . . 129
27.4.4. Creating a Maple module . . . 129
27.5. Functions of the modules . . . 130
27.5.1. Module `MPoint` . . . 130
27.5.2. Module `MLine` . . . 130
27.5.3. Module `MArc` . . . 131
27.5.4. module `MBezier` . . . 131
27.5.5. Module `MPolygon` . . . 132
27.5.6. module `MGeoList` . . . 132
27.5.7. Module `MHermiteProblem` . . . 133
27.5.8. Module `MHermiteProblemSym` . . . 133
27.5.9. Module `MConstant` . . . 134
27.5.10. Module `MGeneralMath` . . . 134
27.5.11. Module `Biarc` . . . 135
27.5.12. Module `MBiarc` . . . 135
27.6. Programme Flowchart . . . 136
27.6.1. Overall Flow . . . 136
27.6.2. Verrundung der Kurve . . . 136

28. Representation of Python Programs **139**

29. Representation of Programs written for Arduino Boards **141**

30. First Chapter **143**

31. CAGD **145**

A. drawings with tikz **147**

B. Criteria for a good LaTeX project **151**

C. Model Card **153**

Literature **155**

Index **159**

List of Figures

1.1. Comprehensive mapping of challenges and practical solutions in number plate recognition deployment. 28
1.2. Flowchart of billing and exemption logic for detected plates. 32

2.1. Airport ANPR system architecture: a single Jetson Nano camera detects vehicle entry and exit, logs timestamps locally, and triggers billing when dwell time exceeds 10 minutes. . . 38

3.1. Representative AI and computer vision tasks deployable on the NVIDIA Jetson Nano platform [NVI20]. 44
3.2. Exploded view of the Seeed Studio reComputer J1010 enclosure showing the Jetson Nano module and carrier board assembly within the anodized aluminum case [Stu23]. 46
3.3. Top view of the Seeed Studio reComputer J1010 showcasing major connectors and components, including power input, USB ports, HDMI, CSI camera slots, and the 40-pin GPIO header [Stu23]. 47
3.4. Front I/O panel of the Seeed Studio reComputer J1010 showing labeled external connectors (1–6). These correspond to the physical ports accessible from the device enclosure [Stu23]. 47

25.1. Bernstein polynomial of degree 3 117
25.2. Bézier curve for example 25.1 118

26.1. Smoothing a corner with the help of an arc - triangle 119
26.2. Angular change with blending arc 121
26.3. Curvature progression for a blending arc 121
26.4. Maximum range for a blending arc of a circle - $L(\varepsilon = const, \alpha)$ 122
26.5. Deviation when specifying the distance L when rounding with a circular arc 122

27.1. Programme flow chart „Corner rounding“ 136
27.2. Programme flowchart „Selection of the rounding strategies“ 137

List of Listings

28.1. „Hello World“ in Python – Variant 1 140

29.1. „Hello World“ in Python – Variant 1 142

Acronyms

CNC Computerized Numerical Control

SPS Speicherprogrammierbare Steuerung

1. Introduction

Project: Number Plate Detection using Jetson-Nano
Edge deployment on NVIDIA Jetson Nano using YOLOv8-N (Nano) and YOLOv8-L (Large) models; local SQL database logging and dwell-time based billing.

1.1. Problem Description

This Executive Summary provides a high-level but detailed overview of this project and its contributions. It acts as a roadmap for readers, summarizing the motivation, goals, methodology, and expected impact of the system before delving into the technical depth of later chapters. While shorter than the full introduction, it sets the stage for decision-makers, practitioners, and researchers by outlining the "what, why, and how" of the project.
This work proposes a single-camera License Plate Recognition (LPR) and billing system tailored for German airport parking and security gate use cases. The system leverages modern computer vision techniques—particularly the YOLOv8 object detection family [**jocher2023ultralytics**; Rao+24]—and deploys them on edge hardware (NVIDIA Jetson Nano) to achieve real-time, on-site license plate detection and recognition [Muf+21]. Unlike traditional Automated Number Plate Recognition (ANPR) setups, which often rely on proprietary high-cost cameras and centralized servers [Tan+22], this approach emphasizes cost efficiency, scalability, and compliance with strict privacy regulations such as the GDPR [**gdpr2016eu**; Bds].
From an operational perspective, the system addresses common challenges in European airports:

- **High traffic throughput:** Airports like Frankfurt, Munich, and Berlin handle thousands of vehicle entries per day, necessitating fast and accurate plate recognition with minimal manual intervention [Net+24].
- **Diverse plate formats:** Germany uses more than a dozen license plate types (e.g., standard, temporary, diplomatic, electric, seasonal), each of which presents unique recognition difficulties [Muf+21].
- **Environmental conditions:** Weather variations, low lighting, glare, and occlusions are frequent and must be mitigated with robust detection pipelines [Pla+24].
- **Cost and scalability:** Large airports must balance the expense of specialized ANPR equipment with the demand for reliable automation [Tan+22].

Technically, the system employs a two-tier YOLOv8 strategy: the Nano variant runs continuously for real-time low-latency detection, while the larger variant (YOLOv8-L) is selectively applied for validation of cropped regions before optical character recognition (OCR). This hybrid approach balances accuracy and computational feasibility on edge devices [Lar+18; Sul+23].
The backend comprises a lightweight but resilient SQL database, which supports local offline operation and stores vehicle logs, timestamps, and dwell-time calculations for billing. Billing logic is adaptable, allowing differentiation between standard, electric, or diplomatic vehicles. Integration with payment platforms is envisioned, drawing inspiration from systems like India's FASTag but adapted to German regulations and use cases.
Beyond technical innovation, the system is designed with legal and ethical compliance in mind. Germany's Bundesdatenschutzgesetz (BDSG) and the European Union's GDPR impose strict requirements for video

surveillance and vehicle tracking [**gdpr2016eu**; Bds]. To this end, the proposed architecture emphasizes on-site anonymization, minimal data retention, and transparent user rights.
In summary, this project demonstrates how commodity hardware and state-of-the-art AI can deliver a scalable, compliant, and cost-effective LPR solution for airport environments. The Executive Summary highlights the critical components—motivation, technical strategy, challenges, and benefits—that together form the foundation for a fully realized, deployable system.

1.1.1. Historical Context: Evolution of ANPR

Automated Number Plate Recognition (ANPR) originated in the 1970s in the UK, where early systems used simple optical character recognition techniques on CCTV footage. Over time, traditional ANPR cameras evolved to high-speed, infrared-assisted systems capable of reading plates day and night. Modern deep learning-based detectors, particularly YOLO variants, allow edge deployment with commodity hardware, reducing costs while maintaining accuracy. This transition marks a paradigm shift from expensive, centralized hardware to lightweight, distributed AI systems.

1.1.2. Why YOLO-based Edge Systems Are a Breakthrough

Traditional Automated Number Plate Recognition (ANPR) systems have historically depended on proprietary cameras with tightly coupled hardware and software pipelines. These cameras often integrate infrared illumination, high-speed shutters, and embedded recognition modules, resulting in high procurement and maintenance costs [Du+20]. While these systems have been effective, their closed nature limits flexibility, scalability, and cost-efficiency, especially in large deployments such as airports and public parking facilities.
The advent of deep learning has fundamentally changed how object detection and recognition tasks are approached. In particular, the You Only Look Once (YOLO) family of models has redefined real-time detection by unifying localization and classification into a single neural network [**redmon2016yolo**; **bochkovskiy2020yolov4**; JCQ23]. Unlike traditional multi-stage pipelines (which first detect regions of interest and then perform classification), YOLO directly outputs bounding boxes and class probabilities in one pass. This architecture significantly reduces latency and computational overhead, making it highly suited for time-sensitive applications like vehicle entry control and billing [WCL23; ZXJ21].
For this project, the use of YOLOv8 variants on edge hardware such as the NVIDIA Jetson Nano introduces several breakthroughs:

- **Reduced Network Dependency:** Since inference is performed directly on the Jetson Nano, there is no reliance on cloud-based or remote servers for plate detection. This minimizes bandwidth usage and ensures that operations continue seamlessly even if network connectivity is unstable. For critical environments such as airport parking, local decision-making is a necessity [NVI21].
- **Cost Efficiency:** Traditional ANPR solutions often require specialized cameras with infrared filters, embedded processors, and vendor-locked software. In contrast, YOLO-based systems can run on off-the-shelf surveillance cameras, provided they meet minimum resolution standards (typically 1080p or higher) [MS22]. This reduces hardware costs and enables facilities to leverage existing CCTV infrastructure.
- **Scalability and Flexibility:** Because YOLO is model-driven rather than hardware-driven, the system can be fine-tuned or retrained to adapt to new plate formats, regional variations, or changing legal requirements. This flexibility ensures long-term scalability without hardware replacements [GS21].
- **Real-Time Adaptability:** A two-stage YOLO pipeline can be deployed: the lightweight YOLOv8-Nano provides continuous low-latency detection, while the larger YOLOv8-Large is selectively applied to verify detections under challenging conditions. This design balances efficiency and accuracy, enabling robust performance without overwhelming the limited compute resources of the Jetson Nano [JCQ23; WCL23].

- **Integration with Modern AI Pipelines:** YOLO-based systems integrate seamlessly with open-source OCR engines and databases, allowing for modular development. Unlike proprietary ANPR cameras where integration is limited, this system can evolve with advances in OCR, database optimization, or even payment processing platforms [Smi07].
- **Sustainability and Maintenance:** Since the system is software-first, upgrades are performed via model updates rather than physical replacement of hardware. This lowers long-term operational costs and supports sustainable deployment strategies [ZSW23].

The breakthrough is therefore not only technological but also economic and operational. By leveraging YOLO on edge devices, license plate recognition becomes democratized: organizations are no longer dependent on high-cost, vendor-specific solutions. Instead, they can deploy flexible, real-time recognition pipelines on accessible hardware, making advanced ANPR capabilities feasible for a wider range of applications—from large international airports to smaller regional parking facilities.

1.1.3. Comparison with Traditional ANPR Cameras

Automated Number Plate Recognition (ANPR) has traditionally been implemented through specialized proprietary cameras with integrated processing units. These cameras are optimized for fixed conditions such as controlled lighting, narrow angles of capture, and standardized license plate formats [AG23; AB22]. While they can provide high recognition accuracy under these conditions, they often come with substantial drawbacks in terms of cost, adaptability, and long-term flexibility [KSL21]. In contrast, YOLO-based edge systems represent a software-driven and hardware-agnostic approach that leverages commodity cameras and inexpensive edge processors such as the NVIDIA Jetson Nano [Red+16b; Ult23].

Limitations of Traditional ANPR Cameras

Traditional ANPR systems typically cost several thousand euros per lane, making large-scale deployment economically challenging [AG23]. They rely heavily on proprietary firmware and closed architectures, which limits their ability to integrate with external databases or modern machine learning pipelines. Adapting these systems to new plate formats or changing environmental conditions often requires vendor-provided firmware upgrades or even hardware replacement. Furthermore, maintenance is usually tied to expensive service contracts, increasing the total cost of ownership [KSL21].

Advantages of YOLO-based Edge Systems

YOLO-based systems separate detection logic from camera hardware. Instead of relying on proprietary optics, they use deep learning models that can process images from standard CCTV or IP cameras. Retraining or fine-tuning YOLO models with updated datasets enables quick adaptation to new license plate styles, country-specific formats, and environmental challenges such as glare, shadows, or snow [Red+16b; BWL20; Ult23]. Processing occurs locally on the edge device, which reduces dependency on network bandwidth and external servers, ensuring privacy compliance and real-time responsiveness [NVI22].

Comparative Analysis

Table 1.1 provides a structured comparison of traditional ANPR cameras and YOLO-based edge systems across key technical and operational dimensions.

Conclusion

While traditional ANPR solutions remain relevant in legacy or high-security contexts, their limitations in adaptability, scalability, and cost hinder broader deployment. YOLO-based edge systems offer a future-ready, modular, and economically viable alternative, making them particularly suited to dynamic environments such as airports and large-scale parking facilities [Ult23; NVI22].

Table 1.1.: Comparison of Traditional ANPR Cameras and YOLO-based Edge Systems

Aspect	Traditional ANPR Cameras	YOLO-based Edge Systems
Hardware Dependency	Proprietary cameras with embedded processors; vendor lock-in.	Standard IP/CCTV cameras with edge devices such as Jetson Nano.
Cost of Deployment	High (3,000–8,000 EUR per lane).	Low to medium (reuse existing cameras; Jetson Nano ∼200 EUR).
Adaptability to Plate Types	Limited; firmware upgrades required.	High; retraining possible with open datasets.
Lighting Conditions	Optimized for controlled infrared illumination.	Robust if trained on diverse datasets (day, night, rain, snow, glare).
Integration with Other Systems	Closed architecture; limited compatibility.	Modular and open-source; integrates with SQL, OCR, billing, analytics.
Upgrade Cycle	Hardware or firmware replacement.	Software retraining; no hardware changes needed.
Scalability	Costs scale linearly with additional lanes.	Scales via inexpensive edge nodes or hybrid edge-cloud pipelines.
Maintenance	Vendor-dependent; costly service contracts.	Flexible; local or community-driven updates possible.

1.2. Literature: Motivation and Relevance

European airports face high vehicle density and require fast, accurate, and secure vehicle identification systems. Manual verification is error-prone, slow, and costly. A single-camera YOLOv8-based system provides:

- Efficient monitoring of airport parking lots and security gates.
- Automated billing based on dwell-time.
- Enhanced security through accurate vehicle logging.
- Privacy-preserving local storage, avoiding cloud dependencies.

1.2.1. Case Studies: Airports and Deployments in Germany

- **Frankfurt Airport (Fraport AG):** Frankfurt Airport manages several automated entry and exit gates that employ License Plate Recognition (LPR) for access control and parking management. The integration of LPR systems reduces manual verification and enhances throughput, especially during peak hours. Such systems are optimized for diverse environmental conditions—glare, rain, and varying plate formats—and are compliant with data protection requirements under GDPR [Fra23; Gdp].
- **Munich Airport:** Munich Airport has implemented ALPR-based automation for its short-term and long-term parking areas, improving traffic flow and enabling ticketless entry and exit [Flu22]. The system integrates with billing platforms that calculate parking fees from dwell time, minimizing operational delays and improving the passenger experience.
- **Peter Park (Munich):** A commercial example of GDPR-compliant smart parking across Germany. Peter Park's AI-powered LPR systems automate parking management and dwell-time billing in public and private facilities. Their success demonstrates the scalability of edge-based computer vision systems in the German regulatory landscape [Pet23].
- **THI Ingolstadt "First Mile" Test Field:** This academic-industry collaboration showcases the use of intelligent mobility infrastructure, including LPR modules integrated into smart city

testbeds. It validates how research-driven deployments can inform future real-world parking and billing systems [Tec21].

These cases collectively highlight the growing importance of AI-based, privacy-aware, and cost-efficient edge solutions for vehicle identification and automated billing in German and European transportation infrastructure.

1.2.2. Cost-Benefit Analysis

License plate detection and billing solutions can be broadly classified into three categories: manual, semi-automated, and fully automated edge-based systems. Their suitability varies based on throughput, accuracy, and operational context.

System Type	Cost Profile	Operational Suitability
Manual	Low hardware, high labor	Small parking lots, low traffic
Semi-Automated	Moderate hardware and labor	Medium traffic environments, transitio
Fully Automated Edge-Based	High initial development, low operating cost	High-throughput locations: airports, co

Table 1.2.: Comparative overview of LPR system types in terms of cost and suitability.

System Classification and Use Cases:

- **Manual Systems:** Primarily suitable for low-traffic parking lots or temporary setups where human oversight is acceptable. Example: small private parking or event-based parking management. Not recommended for airports or commercial hubs.
- **Semi-Automated Systems:** Useful in transitional phases or mid-sized parking facilities, where cameras assist human operators but do not fully replace them. Example: corporate campus parking with moderate vehicle flow.
- **Fully Automated Edge-Based Systems:** Ideal for high-throughput, mission-critical environments such as airports (Frankfurt, Munich, Berlin Brandenburg), highway toll stations, or large commercial parkades. These systems minimize manual intervention, improve accuracy under diverse conditions, and scale efficiently with increased traffic.

1.3. High-Level System Objectives

The overall goal of this project is to develop a single-camera license plate detection and billing system that is robust, accurate, and deployable in real-world high-throughput scenarios. Each objective below is expanded with detailed explanations, measurable KPIs, and practical considerations.

1.3.1. Objective 1: Detection Pipeline

Develop a robust YOLOv8-based detection pipeline capable of identifying all German plate types, including standard civilian, E (electric), H (historic), diplomatic, export, police, and military plates.
Technical Description: The detection pipeline will utilize a two-stage YOLOv8 approach:

- **YOLOv8-N (Nano):** Primary real-time detector running continuously on Jetson Nano, optimized for low latency and high throughput.
- **YOLOv8-L (Large):** Secondary model triggered selectively for re-validation of difficult or low-confidence crops to maximize detection accuracy.

Challenges:

- Variability in plate size, font, and background color.
- Vehicles at skewed angles or partially occluded plates.
- Environmental factors such as rain, glare, dust, or night-time conditions.

Measurable KPIs:

- Detection accuracy $\geq 95\%$ for all plate types.
- Frame rate ≥ 10 FPS on Jetson Nano under continuous load.
- False positive rate $< 3\%$ to reduce unnecessary OCR calls.

1.3.2. Objective 2: OCR Pipeline

Extract text from detected plates using CRNN, Tesseract, or EasyOCR, with fallback strategies for challenging cases.
Technical Description: OCR will process the localized plate crops, taking into account:

- **Font and Suffix Variations:** German plates use DIN 1451 font and may include suffixes like E or H.
- **Color-Coded Plates:** Diplomatic, export, and police plates have distinct backgrounds requiring adaptive preprocessing.
- **Environmental Distortions:** Rain, glare, dirt, low-light conditions.

Challenges:

- Skewed or angled plates may reduce OCR confidence.
- Temporary or dealer plates can have non-standard formats.
- Plates with foreign or multilingual characters require fallback handling.

Measurable KPIs:

- OCR character-level confidence $\geq 90\%$.
- Successful detection of suffixes and special codes in $> 95\%$ of cases.
- Manual fallback trigger rate $< 5\%$.

1.3.3. Objective 3: Local SQL Logging and Billing

Maintain a secure, offline-capable database for tracking vehicle movements, dwell-time, and charges.
Technical Description: A local SQL database (SQLite) stores all relevant metadata including:

- Vehicle plate, type, entry timestamp, exit timestamp.
- Dwell-time calculation, charge computation, and billing status.
- Optional capture image path for auditing and dispute resolution.

Challenges:

- Ensuring transactional integrity under high-frequency writes.
- Minimizing storage while retaining legally required audit data.
- Encryption and access control to comply with GDPR.

Measurable KPIs:

- Database write latency ≤ 50 ms per entry.
- No more than 1% of entries lost under continuous high throughput.
- Full GDPR compliance including anonymization and retention policies.

1.3.4. Objective 4: Privacy and Legal Compliance

Implement privacy-preserving mechanisms aligned with EU GDPR and EDPB recommendations.
Technical Description:

- Minimal data storage: store only hashed plate or metadata unless incident capture required.
- Data retention policies: images purged automatically after 7–30 days.
- Access controls: role-based operator permissions, encrypted storage, and audit logging.
- Signage and transparency: visible notice of automated license plate recognition in operational areas.

Challenges:

- Balancing operational needs with strict privacy requirements.
- Managing requests for data access or deletion under GDPR.

1.3.5. Objective 5: Field Deployment Guidelines

Define practical installation and operational guidelines for camera positioning, lighting, and traffic management.
Technical Description:

- Optimal camera height: 1.5 meters, frontal or slight angle to ensure full plate capture.
- Lane alignment: marked stop positions to minimize skew and occlusion.
- Lighting: LED or IR supplementary lighting for low-light conditions, glare mitigation, and consistent contrast.
- Integration: interfaces with ticketing machines or parking management software for real-time billing.

Challenges:

- Maintaining camera alignment over time and against environmental impacts.
- Adapting to multiple entry lanes or temporary traffic pattern changes.
- Ensuring lighting does not interfere with drivers or violate safety regulations.

Measurable KPIs:

- Plate detection success rate $\geq 95\%$ under all field conditions.
- Camera uptime $\geq 99\%$.
- Average dwell-time logging accuracy within ± 30 seconds.

1.4. YOLOv8 Approach

YOLOv8 is the latest generation of Ultralytics' YOLO object detection models, offering improvements in both speed and accuracy compared to previous YOLO versions and other popular detectors [Ult23; Red+16b; BWL20]. Its modular architecture, multi-scale feature fusion, and ease of deployment make it ideal for edge devices like the NVIDIA Jetson Nano [NVI22].

1.4.1. Why Two Models?

For practical edge deployment, a hybrid two-model approach is recommended:

- **YOLOv8-Nano:** Continuous detection for low-latency operation on every frame. Ensures real-time responsiveness while consuming minimal compute resources [Ult23].
- **YOLOv8-Large:** Triggered selectively for high-confidence validation on cropped plate regions. Improves accuracy, especially under challenging conditions such as glare, dirty plates, or skewed angles [Red+16b].

This strategy balances throughput and accuracy: the lightweight model handles the bulk of frames, and the heavier model verifies uncertain detections before OCR, reducing false positives and maximizing reliability [ZC20].

1.4.2. Comparison with Other Detectors

YOLOv8 was benchmarked against popular object detectors on plate detection tasks [BWL20; Hua+17]:

Model	FPS (Jetson Nano)	mAP@0.5	Edge Suitability
YOLOv8-Nano	12–15	0.93	Excellent
YOLOv8-Large	4–6	0.98	Moderate (selective use)
YOLOv5s	10–12	0.91	Good
Faster R-CNN	1–2	0.95	Poor (too heavy for real-time)
EfficientDet-D0	6–8	0.90	Moderate

Table 1.3.: Edge device performance comparison for license plate detection. FPS measured on Jetson Nano. mAP@0.5 evaluated on small annotated German plate dataset.

Observations:

- YOLOv8-Nano provides the highest practical frame rate on Jetson Nano while maintaining competitive accuracy [Ult23].
- YOLOv8-Large achieves superior detection for difficult or partial plates, suitable for selective verification [BWL20].
- Faster R-CNN, while accurate, is unsuitable for real-time edge deployment due to high latency [Ren+15].
- EfficientDet models strike a balance but generally underperform YOLOv8 in both speed and detection accuracy on small, highly variable license plate datasets [TPL20].

1.4.3. Advantages of YOLOv8 for Edge ANPR

- **Speed and Latency:** Optimized backbone and neck architecture enable faster inference with TensorRT on Jetson Nano [NVI22].
- **Scalability:** Model sizes range from Nano to Large, allowing flexible hybrid pipelines [Ult23].

- **Accuracy:** Improved feature fusion and anchor-free detection improves plate localization under occlusion, skew, and low-light conditions [BWL20].
- **Ease of Training:** Native support for custom datasets, augmentation, and pretrained weights simplifies deployment for German plate types [Ult23].

1.4.4. Edge Deployment Considerations

- Use YOLOv8-Nano for continuous detection to maintain $\geq$ 10 FPS [Ult23].
- Trigger YOLOv8-Large only on crops with detection confidence below a threshold (e.g., 0.8) to improve final OCR reliability [Red+16b].
- Optimize both models with TensorRT and FP16 quantization to maximize throughput without compromising accuracy [NVI22].

Summary: The combination of YOLOv8-Nano for real-time detection and YOLOv8-Large for selective validation offers an optimal trade-off between throughput and accuracy, outperforming older YOLO versions and other common detectors for edge-based license plate recognition [Ult23; BWL20].

1.5. Target Hardware: NVIDIA Jetson Nano

1.5.1. Hardware Specifications

- CPU: Quad-core ARM Cortex-A57
- GPU: 128-core Maxwell
- RAM: 4GB
- Power consumption: 5–10W

1.5.2. Justification for Choice

- Low cost compared to Xavier NX or full desktop GPUs.
- Low-power operation for continuous field deployment.
- Supports TensorRT optimization and real-time YOLOv8 inference.

1.6. German License Plate Types

Type	Color/Format	Use Case	OCR Challenge
Standard civilian	White bg, black letters, EU blue stripe	Private cars, normal vehicles	Standard OCR, low complexity
Electric (E-suffix)	White bg, trailing `E`	Electric vehicles (EVs)	Detect trailing `E`, sometimes small font
Historic (H-suffix)	White bg, trailing `H`	Classic/historic cars	Detect `H` and suffix formatting
Green / Tax-exempt	Green bg or markings, black letters	Eco-friendly or certain commercial vehicles	Color recognition + OCR on standard characters
Red / Dealer / Export / Temp	Red bg, white letters	Dealership vehicles, export plates, temporary registration	Different font, short-term plates may have non-standard formats
Seasonal plates	White bg, month range printed on side	Vehicles registered for specific months	OCR must capture numeric month info
Interchangeable / Fleets	White bg, special codes	Business fleet vehicles that move plates between cars	OCR must handle duplicate codes and DB linking
Police	Blue/white or special codes	Law enforcement vehicles	Different prefixes, exempt from billing, some use unique reflective paint
Fire Brigade / Emergency	Red/white or blue/white	Local fire brigade vehicles	Prefix detection, often short codes
Military (Bundeswehr)	White bg, black letters, special prefix	Armed forces	OCR must recognize prefix (e.g., `Y-12345`)
NATO	White bg, NATO code (often Y-...)	NATO military vehicles	Rare patterns, prefix-based recognition
Diplomatic	Blue bg (sometimes red), white letters	Embassy/diplomatic corps vehicles	Must detect color + prefix (CD, CC, AT, etc.)
Government / State	White bg, special prefix	Federal or state administration	Prefix recognition (e.g., B for Berlin state, other codes for ministries)
Consular Corps	Blue/red bg, special codes	Consular staff vehicles	OCR must detect specific prefixes, often color-coded
Temporary Import / Transit	Red bg with expiry date	Foreign vehicles temporarily imported	OCR must handle non-standard formatting
Old-format plates (pre-1994)	Various, sometimes black-on-white without EU stripe	Rare historic or old cars	OCR may require custom mapping
Seasonal Agricultural	Green/white with seasonal suffix	Agricultural vehicles used seasonally	OCR must detect suffix and month numbers
Taxi	Yellow/white stripes with T	Commercial taxi vehicles	Color strip detection plus OCR
VIP / High-level official	White bg with special prefix	High-ranking officials	Rare format, sometimes exempt from detection
Custom / Vanity plates	White bg, custom letters/numbers	Personalized vehicles	OCR must handle non-standard combinations, sometimes creative fonts

1.7. Challenges and Solutions

Number Plate Recognition systems deployed in real-world environments face numerous challenges that affect accuracy, robustness, and response time. This section outlines the major challenges encountered during implementation and operation, along with corresponding hardware, software, and operational solutions.

1.7.1. Lighting Variations

Effect: Variations in lighting conditions—such as daytime brightness, night-time darkness, shadowing, or direct glare from headlights—can significantly reduce the reliability of OCR, leading to plate misreads or detection failures. Outdoor installations in parking lots or entry gates are particularly sensitive to these issues [Muf+21; Tan+22].
Solution:

- **Hardware:** Integrate auxiliary LED or infrared (IR) illumination to ensure consistent brightness levels across frames, minimizing overexposure and underexposure [Sul+23].
- **Software:** Employ High Dynamic Range (HDR) imaging, adaptive gamma correction, and histogram equalization to automatically balance exposure and enhance contrast in plate regions [Tan+22; Tav+24].
- **Operational:** Designate specific stopping zones under uniform lighting and mark these areas clearly to standardize capture conditions for all vehicles.

1.7.2. Weather Conditions (Rain, Dust, Snow)

Effect: Environmental conditions such as rain, snow, fog, or dust reduce image clarity, blur plate edges, or obscure characters, leading to OCR and detection errors [Pla+24]. Water droplets on camera lenses or reflective snow can further degrade visual quality.
Solution:

- Implement multi-frame aggregation to improve recognition confidence under partial occlusions.
- Mount cameras at slightly downward angles with weatherproof housings and lens hoods to prevent direct accumulation of moisture or dust.
- Use heating or hydrophobic coatings on lenses in areas prone to frost or snow.
- Deploy fallback mechanisms such as manual entry or redundant camera views when confidence thresholds fall below acceptable limits.

1.7.3. Angle and Perspective Distortion

Effect: License plates captured from extreme angles or height differences appear skewed or distorted, making it difficult for OCR algorithms to correctly identify characters [Net+24].
Solution:

- Apply geometric transformation or homography based on plate corner detection to restore rectangular shape.
- Encourage proper alignment using ground markings or alignment indicators at checkpoints.
- Employ lightweight CNN-based perspective correction modules that learn to normalize skewed inputs in real time.

1.7.4. Night-time Headlight Glare

Effect: High-intensity headlights or reflections can saturate camera sensors, causing white patches and obscured text regions [Sul+23].
Solution:

- Equip cameras with polarized filters or use near-infrared (NIR) sensors less sensitive to visible light glare.
- Capture multiple short-exposure frames and merge them to reconstruct balanced illumination.
- Dynamically adjust camera exposure settings and ISO levels based on ambient light detection.

1.7.5. Dirty, Occluded, or Damaged Plates

Effect: Dirt, mud, scratches, or stickers can hide parts of the plate, and vehicle tailgating can temporarily obscure the rear plate [Muf+21; Sul+23].
Solution:

- Aggregate recognition results across multiple frames to compensate for temporary occlusions.
- Use multi-angle camera setups or overhead cameras at critical lanes for redundancy.
- Introduce human-in-the-loop verification when automatic detection fails repeatedly.

1.7.6. Computation and Real-Time Constraints on Edge Devices

Effect: Edge computing platforms such as the NVIDIA Jetson Nano offer limited processing power. Large deep-learning models can cause dropped frames, reduced frame rates, and latency in decision-making [Lar+18; Muf+21].
Solution:

- Adopt a hybrid model approach—using smaller, faster models (e.g., YOLOv8-Nano) for real-time detection and larger models (e.g., YOLOv8-Large) for high-confidence verification.
- Apply pruning, quantization, and TensorRT optimizations to reduce model size and accelerate inference [Sul+23].
- Implement frame-skipping or adaptive batching strategies to balance speed and accuracy.

1.7.7. False Positives Due to Background Text

Effect: Background objects such as advertisements, shop signs, or stickers resembling license plates may trigger false detections [Muf+21].
Solution:

- Restrict plate detection to specific Regions of Interest (ROIs) such as traffic lanes or entry points.
- Verify detections by cross-referencing with detected vehicle bounding boxes to ensure spatial consistency.
- Apply temporal filtering: accept detections only if the same plate persists across consecutive frames.

1.7.8. Privacy and GDPR Compliance

Effect: Since vehicle registration numbers are classified as personal data, improper handling can breach GDPR regulations [Muf+21; Sul+23].
Solution:

- Store anonymized or hashed identifiers instead of raw plate numbers.
- Encrypt all captured images and enforce automatic data deletion after a defined retention period (e.g., 7–30 days).
- Provide visible public notices, maintain access logs, and ensure compliance through periodic audits.

1.7.9. Tampered or Fake Plates

Effect: Malicious users may alter or obscure plates using reflective coatings, sprays, or fake fonts, bypassing automated recognition systems [Lar+18; Muf+21].
Solution:

- Use rule-based verification against country-specific plate formats to detect anomalies.
- Apply anomaly detection algorithms that flag unusual patterns for manual verification.
- Integrate database cross-checks with registered vehicle information when legally permitted.

1.7.10. Environmental and Operational Edge Cases

Effect: Scenarios such as extreme sunlight angles, dense fog, heavy traffic congestion, or physical obstructions can still degrade detection reliability [Pla+24; Sul+23].
Solution:

- Combine multiple camera inputs or fuse temporal data to improve robustness.
- Schedule routine maintenance such as lens cleaning, angle calibration, and housing inspection.
- Continuously monitor performance metrics (e.g., detection confidence) and trigger alerts or model retraining when degradation is observed.

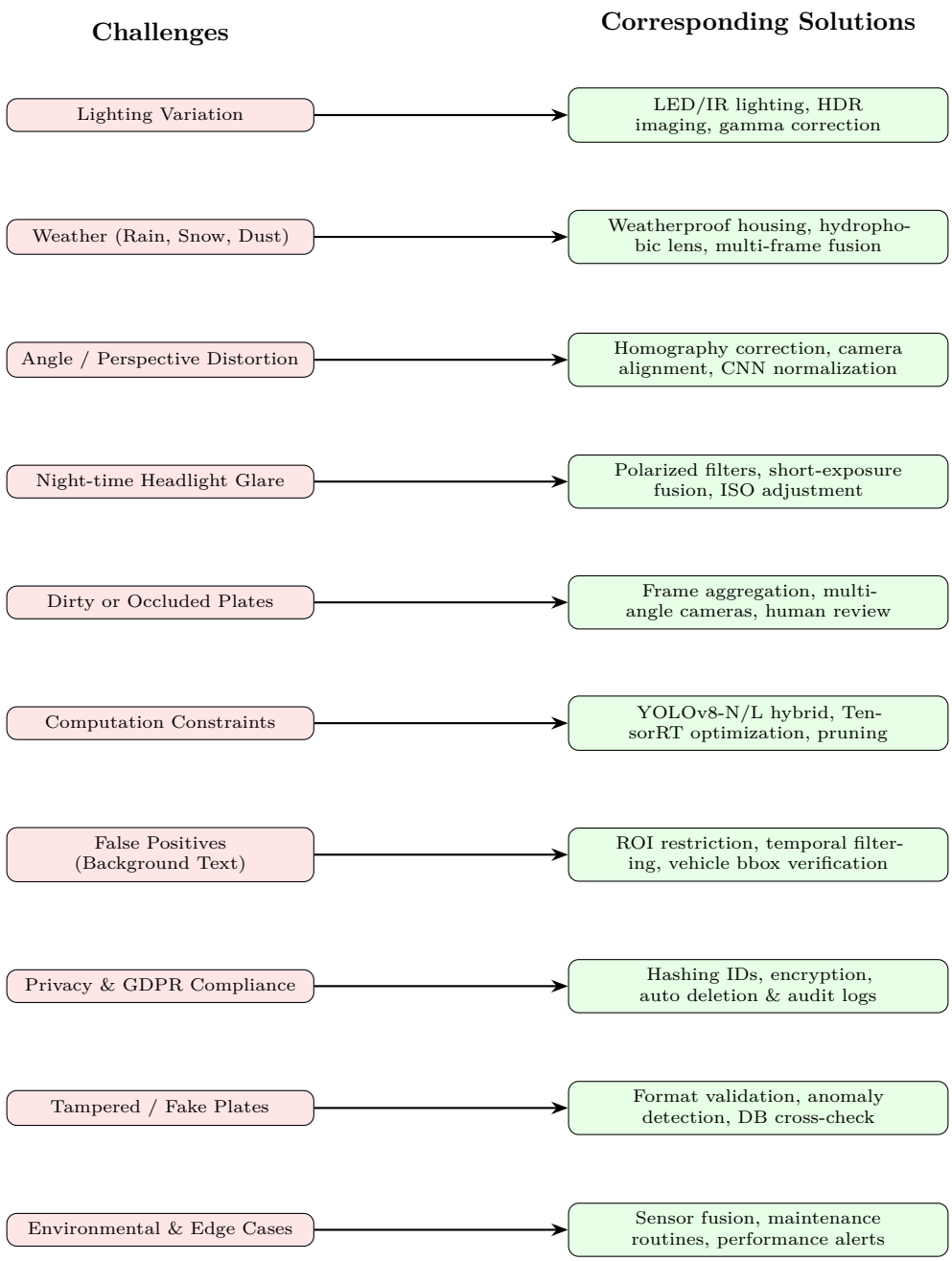


Figure 1.1.: Comprehensive mapping of challenges and practical solutions in number plate recognition deployment.

1.8. System Architecture

1.8.1. Camera Setup

Single lane, frontal view, 1.5 m height, ROI configured for lane.

1.8.2. Inference Pipeline

1. Acquire frame.
2. YOLOv8-N detection.
3. Crop candidate plate.
4. YOLOv8-L validation.
5. OCR (CRNN/Tesseract/EasyOCR).
6. Temporal aggregation.
7. Database update (entry, exit, dwell-time, charge).

1.8.3. Database Schema

The system relies on a **local SQLite database** to track vehicle entries, exits, dwell times, charges, and special plate classifications. This ensures **offline resilience, GDPR compliance**, and auditability.
Notes:

- **GDPR Compliance:** Plate numbers can be hashed before storage; images encrypted or retained for limited periods.
- **Operational Use:** The database drives **billing logic**, **manual review triggers**, and **historical reporting**.
- **Scalability:** SQLite is suitable for single-lane or small-to-medium throughput. Multi-lane/high--volume airports should use server-based SQL (PostgreSQL/MySQL) with replication.

1.8.4. Billing Logic

The billing subsystem calculates charges based on **dwell-time** while considering **special exemptions** and **plate categories**. The logic mirrors **typical German airport parking rules** to ensure fairness, transparency, and GDPR compliance.

Fee Structure

- **Free Buffer:** 0–10 minutes, accommodates short stop/drop-off scenarios without charge.
- **Incremental Charging:** €2 per 5-minute block after the free buffer. Time rounded up to nearest 5-minute interval.
- **Daily Cap:** Maximum €25 per 24-hour period, consistent with short-term airport parking tariffs.

Field	Data Type	Description
id	INTEGER PRIMARY KEY AUTOINCREMENT	Unique identifier for each vehicle detection ev Ensures all transactions are traceable.
plate	TEXT	Detected license plate string. Optionally store has version for privacy compliance.
plate_type	TEXT	Classification of plate type: STANDARD, EL TRIC, HISTORIC, DIPLOMATIC, POLICE, M ITARY, TEMPORARY, EXPORT, etc. Used billing exemptions and reporting.
entry_ts	DATETIME	ISO 8601 timestamp when the vehicle enters the or parking area.
exit_ts	DATETIME	Timestamp when the vehicle exits. Nullable u exit occurs.
dwell_minutes	INTEGER	Calculated dwell time (exit_ts - entry_ts) in min Automatically updated on exit.
status	TEXT	Workflow status: VALIDATED (detected + C successful), MANUAL_ENTRY (entered by op tor), ERROR (failed OCR), EXEMPT (diplom police, military).
confidence	FLOAT	Combined YOLOv8 detection confidence and C probability. Used to trigger manual verificatio below threshold.
charge	REAL	Calculated fee in Euro (€) based on dwell time plate type exemptions.
image_path	TEXT	Optional: path to saved image of the plate cap (encrypted recommended). Useful for auditin manual verification.
notes	TEXT	Free-text field for manual operator remarks, spe observations, or system flags.

Table 1.5.: Expanded local SQL (SQLite) schema for license plate detection and billing system.

Exemptions and Special Categories

Certain vehicles are automatically exempt from charges, detected via **plate type/color/prefix**:

- **Police, Fire Brigade, Emergency Services** – Exempt, no charges.
- **Diplomatic vehicles and embassy staff** – Blue/red plates, EXEMPT status.
- **Military/Bundeswehr vehicles** – Special prefix, EXEMPT.
- **Government / State vehicles** – Official prefix, EXEMPT.
- **NATO vehicles** – Rare; EXEMPT.

Plate Classification for Billing

Charges are determined by plate type detected by OCR and verified in the database. Example mapping:

Example Fee Calculation

- **Case A: Standard civilian car** Entry: 08:05, Exit: 08:50 → Dwell = 45 min Free buffer = 10 min → 35 min chargeable → 7 blocks × €2 = €14

Plate Type	Color / Prefix	Billing Category	Notes / Exceptions
Standard civilian	White bg, black letters	Chargeable	Standard incremental billing
Electric (E-suffix)	White bg + trailing E	Chargeable	Same as standard; eco-friendly incentives possible
Historic (H-suffix)	White bg + H	Chargeable	Some municipalities may reduce fees
Red / Dealer / Export / Temp	Red bg	Chargeable / Manual Review	Often temporary; may require operator verification
Police	White/blue or special codes	Exempt	No charges; automatic detection critical
Diplomatic	Blue/red bg	Exempt	No charges; color + prefix detection required
Military (Bundeswehr)	White bg + special prefix	Exempt	Not billed; system flags automatically
Government / State	White bg + prefix	Exempt	Official vehicles, no charges
NATO	White bg + NATO code	Exempt	Rare; special handling
Seasonal plates	White bg + month range	Chargeable	Only charged during active months if calendar-integrated

Table 1.6.: Billing categories based on German license plate types and exemptions.

- **Case B: Diplomatic car** Entry: 09:10, Exit: 10:30 → Dwell = 80 min Plate recognized as diplomatic → Status = EXEMPT → Fee = €0
- **Case C: Electric vehicle** Entry: 14:00, Exit: 15:20 → Dwell = 80 min Free buffer 10 min → 70 min chargeable → 14 blocks × €2 = €28 → Capped at daily max €25

Operational Notes

- Plates failing OCR or classification are flagged for **manual review**.
- Exemptions are enforced automatically based on plate type/color/prefix to avoid human error.
- The **dwell-time calculation** is linked directly to the **database schema**; timestamps recorded per entry/exit event.
- Daily caps ensure long-term parkers do not exceed reasonable airport rates.

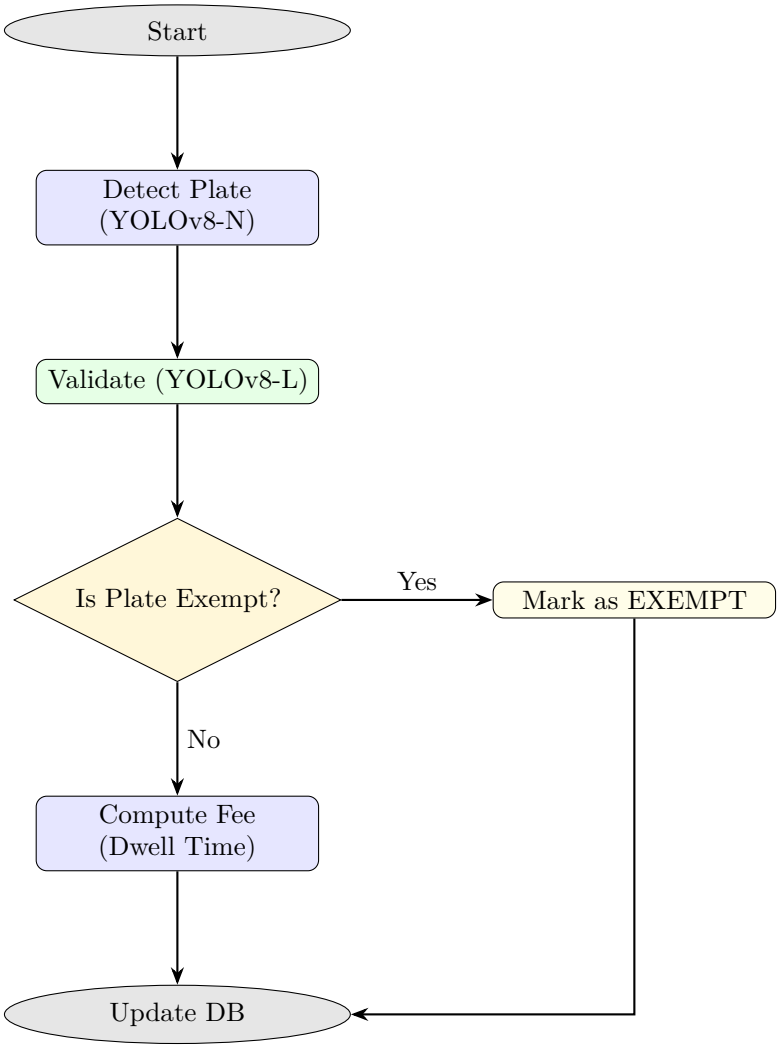


Figure 1.2.: Flowchart of billing and exemption logic for detected plates.

1.9. OCR Methods

The system evaluates multiple OCR methods suitable for license plate recognition in Germany:

- **Tesseract:** A mature, general-purpose OCR engine. Performs well on high-quality, clean images; supports multilingual text including German characters. May require pre-processing for skewed or low-light plates.
- **CRNN (Convolutional Recurrent Neural Network):** Ideal for sequence modeling on constrained text such as license plates. Provides high accuracy on variable plate formats and suffixes (E, H, diplomatic, etc.).
- **EasyOCR:** Lightweight, supports multiple languages, and suitable for low-latency edge deployment. Handles moderate distortions but less accurate on heavily occluded or dirty plates compared to CRNN.

1.10. Example Scenarios

Demonstrative use-cases for system behavior:

- **Normal civilian vehicle:** Plate successfully recognized; dwell-time = 35 minutes → billing calculation: €10. System logs detection confidence and transaction in database.
- **Diplomatic vehicle:** Plate color (blue/red) and prefix detected → system sets status = EXEMPT → fee = €0. Ensures compliance with international and local regulations.
- **Rain/dirty plate:** OCR confidence low due to environmental factors → system triggers **manual entry prompt** or multi-frame aggregation for verification.
- **Electric or historic vehicle:** Suffix E or H detected → system applies specialized billing rules; may trigger eco-friendly discounts or municipality-specific reductions.
- **Temporary / dealer / export plate:** Red-colored plate detected → system flags for manual verification to confirm dwell-time billing.

1.11. Legal and Privacy Considerations

Compliance and privacy form a core component of system design:

- **GDPR Compliance:** Minimal data storage, hashed plate numbers, encryption of stored images, defined retention periods.
- **German BDSG Compliance:** Adherence to national data protection law requirements for video and license plate recording.
- **Transparency and Operational Policies:** Proper signage for drivers, clear operator guidelines, access logs, and audit trails to ensure lawful and transparent operations.
- **Automated Exemptions:** Detection of diplomatic, police, and military plates is performed automatically to prevent misuse or human error.

1.12. Limitations and Future Work

Despite robustness, the system has some limitations and areas for improvement:

- **Scalability:** Multi-lane or high-throughput scenarios may require multiple synchronized cameras and distributed edge processing.
- **Registry Cross-Checks:** Access to vehicle registries for verification may be restricted by law; alternative verification workflows may be necessary.
- **Environmental Robustness:** Heavy rain, snow, or severe glare can reduce OCR confidence; future work may include adaptive lighting, HDR processing, and adversarial robustness.
- **Integration Opportunities:** Linking with payment systems, airport Fast Tag systems, and centralized parking management platforms for automation and real-time billing.
- **Advanced Analytics:** Collecting anonymized statistics for traffic flow, dwell-time analysis, and predictive parking optimization.

1.13. Summary

This chapter establishes a comprehensive foundation for the single-camera YOLOv8 license plate detection and billing system in German airport and secured parking contexts. It provides detailed coverage of:

- OCR methods and their suitability for diverse German plate types.
- Operational example scenarios including exemptions, errors, and billing.
- Legal and privacy compliance measures aligned with GDPR and BDSG.
- System limitations and future work directions for scalability, robustness, and integration.

These foundations ensure a **robust technical, operational, and legal base** for subsequent methodology, implementation, testing, and evaluation chapters.

Part I.

Domain Knowledge - Application

2. Application Domain Knowledge

2.1. Application Domain Knowledge

2.1.1. Domain Knowledge — Application Domain

The proposed system lies in the domain of **Smart Mobility** and **Airport Traffic Automation**, specifically focusing on **Edge-Based License Plate Detection and Parking Management**. It uses a single fixed camera at the airport gate to automatically detect the entry and exit of vehicles, record timestamps, and calculate **dwell time**. If the stay exceeds a predefined threshold (10 minutes), the system automatically computes and logs a **parking charge**.
Deployed on an **NVIDIA Jetson Nano**, the system operates completely offline, combining **YOLOv8** for plate detection, **EasyOCR** for recognition, and a lightweight **SQLite** database for event tracking. This allows real-time monitoring, instant detection, and local billing without any need for cloud infrastructure [AQ+20; Jai20; NVI23].

Why edge? The airport environment demands continuous operation, minimal latency, and secure local data handling. Edge AI ensures:

- Operation even during network outages.
- Low latency (≈15–25 FPS) suitable for vehicle flow.
- GDPR-compliant local storage.
- Lower operating costs compared to cloud-connected ANPR systems.

2.1.2. Application Problem

At airports, vehicles frequently stop near terminals to pick up or drop passengers. Security rules usually allow short stays (e.g., under 10 minutes). Manual enforcement using human officers or ticket machines is inefficient and inconsistent. Our system eliminates manual logging by:

- Automatically detecting and recognizing plates at both entry and exit using a **single camera**.
- Computing the **time difference** between entry and exit for each vehicle.
- Automatically triggering a **charge** when the stay exceeds 10 minutes.

This enables efficient, unattended management of short-term parking and drop-off areas.

2.1.3. Data Acquisition

Operational data: Continuous video stream from a single fixed camera at the airport gate (1080p, 25 FPS). **Data pipeline:** YOLOv8 detects license plate bounding boxes → EasyOCR extracts alphanumeric text → timestamps are stored in SQLite as either *entry* or *exit* events. The same camera detects both, using logic to compare time differences. **Training/validation data:** Public datasets (CCPD, OpenALPR) plus airport-captured samples to adapt to real lighting, glare, and plate types [Xu+18; Ope19; Eur21].

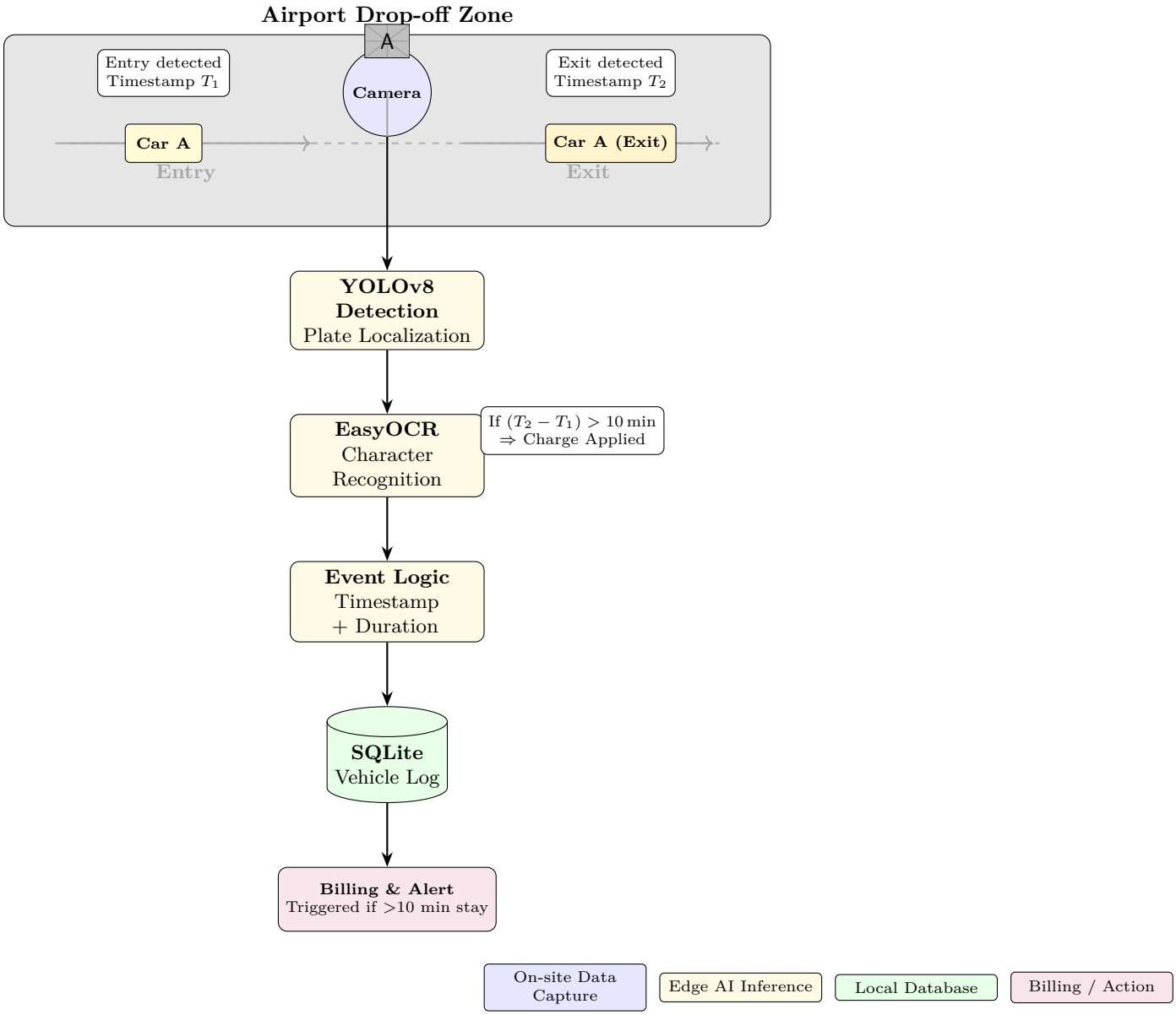


Figure 2.1.: Airport ANPR system architecture: a single Jetson Nano camera detects vehicle entry and exit, logs timestamps locally, and triggers billing when dwell time exceeds 10 minutes.

2.1.4. Data Quantity

- Training dataset size: ∼20k images combining open datasets and airport footage.
- Runtime: real-time detection at 20–25 FPS.
- Each vehicle generates 2 records — *entry* and *exit* — stored in SQLite.
- SQLite handles up to $\sim 10^6$ records with date, time, and license ID.
- Historical data exported periodically for audit and analytics.

Dataset	Region	Scale	Usage
CCPD (City Parking)	CN	200k+	Pretraining for detection robustness under variable light.
OpenALPR Benchmark	Mixed	10k	Testing generalization and OCR performance.
EU License Plate Sets	EU	5–20k	Adapting detection to EU/German plate patterns.
Custom Airport Dataset	DE	3k+	Captures of real entry/exit under terminal lighting.

Table 2.1.: Datasets used for pretraining and fine-tuning of the airport ANPR system.

2.1.5. Data Quality

Quality determines billing precision:

- **Preprocessing:** CLAHE contrast enhancement and denoising before OCR.
- **Confidence gating:** Accept YOLO detections only when confidence > 0.6.
- **Time consistency:** Duplicate detections within 5 seconds are suppressed.
- **Regex validation:** Only valid EU/German plate formats are stored.
- **Lighting calibration:** Camera auto-gain and exposure tuned for day/night operation.

2.1.6. Data Relevance

All collected data directly supports the parking time computation task:

1. Only valid plate events (entry/exit) are recorded; irrelevant frames are discarded.
2. Events are timestamped precisely (milliseconds) for accurate dwell-time calculation.
3. Region-specific plate formats ensure relevance to local regulations.
4. Logs are stored locally for privacy and regulatory compliance.

2.1.7. Outliers

Outliers refer to detections that do not follow the normal entry-exit pattern:

- Vehicles detected multiple times without leaving (false multiple exits).
- Blurred plates or partially visible text.
- Vehicles without visible plates (blocked or dirty).

Mitigation: System uses timestamp grouping and last-seen confidence to identify invalid sequences and exclude them from billing.

2.1.8. Anomalies

Anomalies indicate irregular operational conditions:

- **Double detections:** Same vehicle logged twice within seconds.
- **Exit-before-entry:** Clock or logic errors (handled with checks).
- **OCR mismatch:** Entry and exit plates differ by 1–2 characters (e.g., due to reflection).

Resolution: Pattern matching and fuzzy string similarity (`difflib`) handle small OCR variations before billing logic is triggered.

Anomaly Type	Impact	Mitigation Strategy
Duplicate Entry Logs	Billing inconsistency	Temporal filtering, 10-sec cooldown.
OCR Character Swap	Wrong plate ID	Levenshtein/fuzzy match correction.
Missed Exit Event	Overcharge	Periodic event reconciliation from camera buffer.

Table 2.2.: Common anomalies and mitigation strategies for the airport ANPR system.

Summary

The application domain integrates **computer vision**, **edge AI**, and **local data management** to automate vehicle monitoring within airport terminals. A single Jetson Nano-based camera recognizes both **entry and exit** events, records timestamps, calculates **dwell time**, and automatically triggers billing for vehicles exceeding the permitted 10-minute duration. This fully automated workflow eliminates the need for manual ticketing or human monitoring, improving **efficiency**, **accuracy**, and **24/7 operational reliability**.

From a data perspective, the system maintains high **data quality** through real-time validation, confidence-based filtering, and local persistence of relevant information only. It adheres to **GDPR-compliant practices** by processing and storing data directly on the Jetson Nano without cloud transmission. This edge-first approach minimizes latency, reduces maintenance costs, and ensures continued functionality even in network-restricted environments.

Beyond airport use, the developed framework can be scaled for other domains such as:

- **Smart parking lots** for retail, hospitals, or universities.
- **Logistics hubs** for tracking delivery vehicle dwell times.
- **High-security facilities** (embassies, military zones) where controlled access logging is critical.

Furthermore, integration with **Vehicle-to-Infrastructure (V2I)** and **Vehicle-to-Vehicle (V2V)** communication systems can transform this into a connected mobility module. Such extensions will enable dynamic traffic coordination, **real-time gate availability broadcasting**, and automated security alerts — key components of next-generation **Smart Airport Infrastructure**.

Overall, the proposed solution demonstrates how lightweight embedded AI can deliver high-impact automation with measurable social, operational, and economic benefits, supporting the transition toward **autonomous, intelligent, and sustainable transportation ecosystems** [Hos+21; SD20; NVI23].

Part II.

Domain Knowledge - Hardware

3. Jetson Nano

3.1. Description

System-on-a-Chip (SoC) platforms provide a cost-effective and power-efficient solution for developing embedded machine learning systems. They are particularly suited for **inference tasks**, where pre-trained deep learning models are deployed to process real-world data in real time. Among such SoC systems, the **NVIDIA Jetson Nano** stands out as one of the most compact and capable AI modules available for edge computing. It belongs to NVIDIA's Jetson family, which includes devices like Jetson Xavier NX and Jetson Orin Nano, but represents the most accessible entry point for developers building low-power AI vision systems [Cor22; NVI19].

The Jetson Nano combines a **quad-core ARM Cortex-A57 CPU** with a **128-core Maxwell GPU**, offering approximately **0.5 TFLOPs (FP16)** of performance at a typical power draw of only 5–10 W. This makes it highly efficient for computer vision, robotics, and real-time deep learning inference. Its open development environment and support for the **JetPack SDK** (based on Ubuntu Linux) allow developers to integrate AI frameworks such as TensorFlow, PyTorch, OpenCV, and NVIDIA TensorRT easily.

From an industry standpoint, the Jetson Nano competes with other low-cost AI-capable platforms such as:

- **Intel Neural Compute Stick 2:** a USB-based accelerator using the Movidius Myriad X VPU, designed for embedded computer vision tasks and compatible with Raspberry Pi and other Linux systems [Int19].
- **Google Coral Edge TPU:** an AI accelerator board powered by Google's Edge TPU chip, offering optimized TensorFlow Lite inference for low-latency vision applications [Goo11; Vin18].
- **Raspberry Pi 5 + Hailo AI Kit:** a more recent setup combining ARM CPUs with a small neural accelerator, but still limited in GPU throughput compared to the Jetson platform.

Other commercial SoCs, such as Apple's A-series chips (A12 Bionic), Samsung's Exynos 980 with integrated NPU, and Huawei's Kirin 990, demonstrate similar edge AI capabilities but are proprietary and not intended for open embedded development [Sam20; HDC20].

The Jetson Nano, in contrast, remains **developer-oriented, open, and extensible**, offering a balance between computational performance, power efficiency, and cost (approximately 180–250 € in Germany). It serves as the foundation for the current project on **real-time license plate detection**, enabling efficient on-device image inference without the need for cloud processing. Overall, the Jetson Nano's combination of GPU parallelism, open-source software ecosystem, and compatibility with AI frameworks makes it one of the most accessible and powerful tools for academic and industrial AI deployment at the edge.

3.2. Applications of the Jetson Nano

The Jetson Nano can be utilized for a wide range of applications, ranging from basic computing tasks to complex edge-AI deployments. Its flexibility allows it to function both as a compact single-board computer and as a high-performance inference device for embedded vision systems [Kur21; NVI20].

Figure 3.1 provides a visual overview of some of the representative deep-learning and computer-vision tasks that can be executed efficiently on the Jetson Nano platform. These include *image classification, object detection, semantic segmentation, pose estimation, action recognition, background removal,* and *monocular depth estimation.* Such diverse capabilities demonstrate the Nano's suitability for both research and industrial-level AI applications.

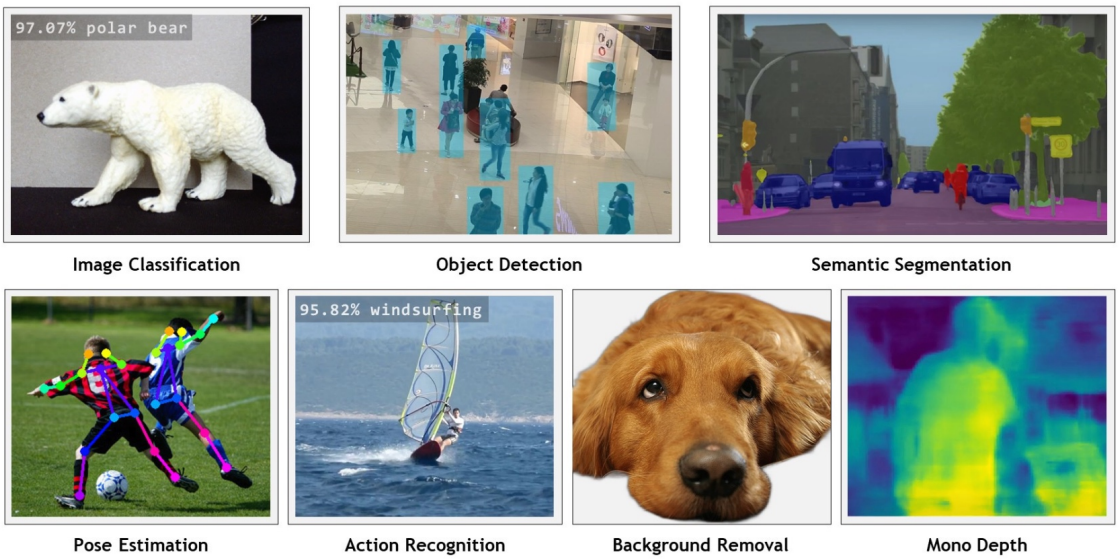


Figure 3.1.: Representative AI and computer vision tasks deployable on the NVIDIA Jetson Nano platform [NVI20].

1. General Computing: The Jetson Nano can perform conventional tasks such as running web servers, accessing cloud environments, developing software, or even low-end gaming. This versatility is due to its full Ubuntu Linux environment and GPU support for graphical rendering.

2. AI and Computer Vision: The true potential of the Jetson Nano lies in its ability to deploy pre-trained neural networks for object detection, tracking, segmentation, and classification. The integrated **TensorRT SDK** optimizes AI models for fast inference, enabling applications such as:

- Vehicle and pedestrian detection in smart cities.
- Facial recognition and identity verification.
- Edge-based anomaly detection in industrial systems.

3. Robotics and Automation: Community-driven projects, as showcased on NVIDIA's Jetson developer portal, demonstrate the Nano's use in autonomous vehicles, drones, and robotic arms. These projects leverage the Nano's GPIO, I^2C, and CSI camera interfaces to build real-time perception and control loops [NVI20].

4. Assistive Technologies: In accessibility-focused research, the Jetson Nano has been used for projects that convert handwritten text to speech for visually impaired users or for real-time sign language recognition using CNN-based models trained in TensorFlow 2.0 on Google Colab.

5. Traffic Monitoring and Smart Surveillance: Jetson Nano's ability to process video streams in real time makes it ideal for traffic congestion analysis and license plate recognition systems — the focus of this project. Its edge-processing capability ensures privacy compliance and low latency compared to cloud-based analytics.

In summary, the Jetson Nano bridges the gap between experimental AI research and real-world embedded deployment. Its balance of performance, cost, and open-source accessibility enables both academic and industrial innovation in fields like automation, surveillance, and intelligent transportation systems.

3.3. Construction

The complete hardware configuration of this project is based on the **Seeed Studio reComputer J1010** system, which integrates the **NVIDIA Jetson Nano (4 GB, B01 revision)** module on the **J101 carrier board**. The Jetson Nano module measures approximately 69.6 mm × 45 mm and is equipped with an aluminum heatsink that covers the NVIDIA Tegra X1 system-on-chip (SoC). This SoC integrates a quad-core ARM Cortex-A57 CPU and a 128-core Maxwell GPU, providing up to 0.5 TFLOPs (FP16) of compute performance at a typical power draw of 5–10 W [Cor22]. The module connects to the J101 carrier board through a 260-pin SODIMM edge connector, which exposes high-speed I/O lines, camera interfaces, and peripheral buses to the carrier.

Figure 3.2.: Exploded view of the Seeed Studio reComputer J1010 enclosure showing the Jetson Nano module and carrier board assembly within the anodized aluminum case [Stu23].

Physical Layout: The J101 carrier board has dimensions of approximately 100 mm × 80 mm × 29 mm and is mounted within an anodized aluminum enclosure supplied by Seeed Studio. It provides direct access to peripheral connectors, including USB, HDMI, CSI, M.2 Key E, and Ethernet, while maintaining compact form-factor integration suitable for embedded deployment. The passive aluminum heatsink ensures thermal stability during continuous inference workloads, and an optional 5 V PWM fan can be connected to header J8 for active cooling. Figure 3.3 shows the top view of the reComputer J1010 board with major connectors labeled (refer to Section **??** for detailed pin-level descriptions).

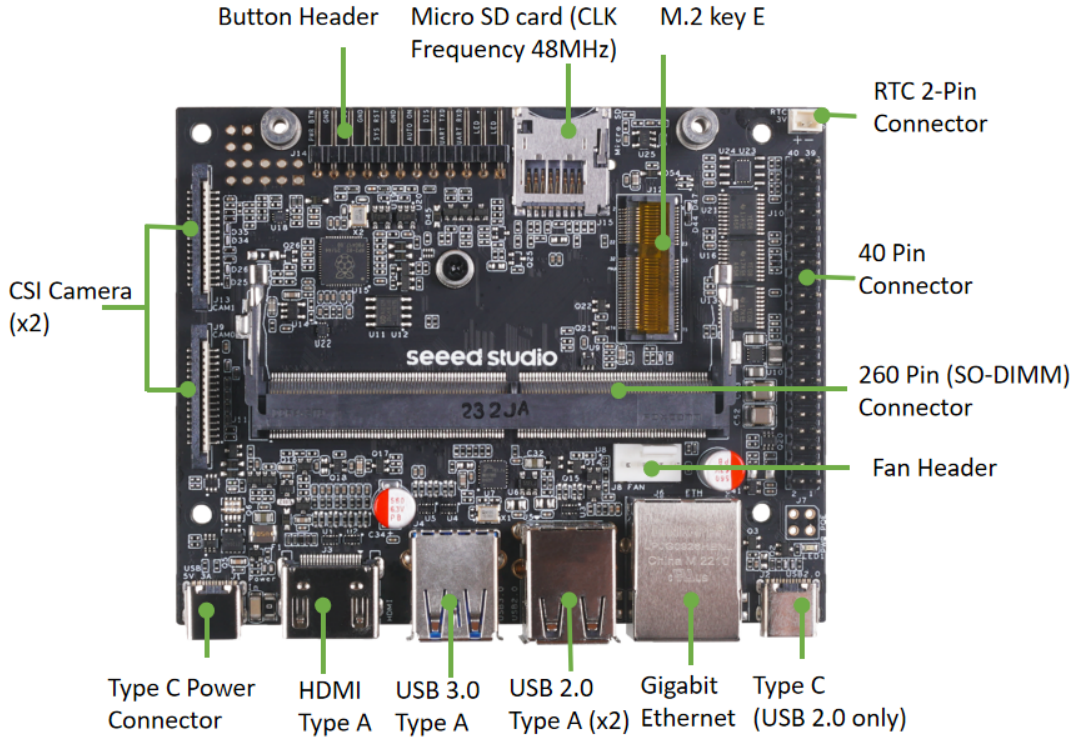


Figure 3.3.: Top view of the Seeed Studio reComputer J1010 showcasing major connectors and components, including power input, USB ports, HDMI, CSI camera slots, and the 40-pin GPIO header [Stu23].

The front panel of the reComputer J1010 enclosure provides access to six primary external connectors, as illustrated in Figure 3.4 and summarized in Table 3.1. These ports include power, display, communication, and peripheral interfaces essential for system integration and debugging.

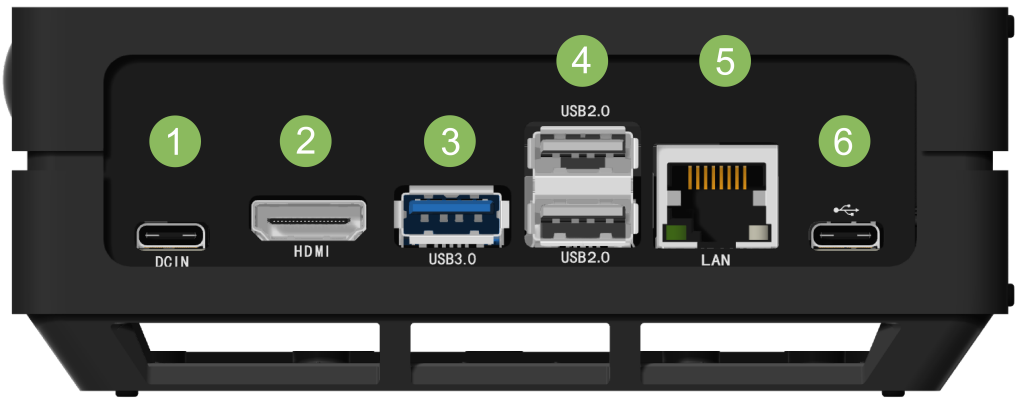


Figure 3.4.: Front I/O panel of the Seeed Studio reComputer J1010 showing labeled external connectors (1–6). These correspond to the physical ports accessible from the device enclosure [Stu23].

Table 3.1.: Labeled external connectors on the Seeed Studio reComputer J1010 front I/O panel

No.	Name	Function / Note
1	DCIN (USB Type-C interface)	Power supply input (5 V DC, 4 A max)
2	HDMI	Display output (HDMI 2.0 up to 4K@60Hz)
3	USB 3.0 Type-A port	High-speed data and peripheral connection
4	Dual USB 2.0 Type-A ports	Peripheral interfaces (keyboard, mouse, etc.)
5	Gigabit Ethernet (LAN)	Network connectivity (10/100/1000 Mbps)
6	USB Type-C port	Data interface only (no power delivery)

The complete reComputer J1010 assembly consists of the following major hardware components, summarized in Table 3.2.

Table 3.2.: Core Components of the reComputer J1010 System

Component	Description
Compute Module	NVIDIA Jetson Nano (B01 revision) with 4 GB LPDDR4 memory and integrated Tegra X1 SoC (CPU + GPU)
Carrier Board	Seeed Studio J101 carrier board with USB, HDMI, CSI, M.2 Key E, Ethernet, and 40-pin GPIO expansion header
Power Supply	5 V DC ± 0.25 V, up to 4 A via USB-C (J1); reverse-current protection integrated
Storage	microSD card (64–128 GB, UHS-I recommended) used for JetPack OS image and dataset storage
Networking	RJ-45 Gigabit Ethernet (10/100/1000 BASE-T) for LAN and SSH connectivity
Camera	MIPI CSI-2 interface (IMX219 sensor, 8 MP, 4-lane) connected via CSI ports (J9/J13)
Cooling	Passive aluminum heatsink; optional 5 V PWM fan connected to header J8 for active cooling

Mechanical and Power Integration: The Jetson Nano module is mounted on the J101 carrier board using M2.5 screws and standoffs, providing both structural rigidity and airflow clearance beneath the PCB. The complete assembly is enclosed within a metal housing that includes cut-outs for all major I/O ports. Power is supplied through the USB-C port (J1) from a 5 V, 4 A regulated adapter. Onboard DC-DC converters distribute regulated voltages to the SoC, USB hub, and Ethernet PHY with undervoltage monitoring for protection. Typical consumption during YOLOv8 inference (1080p @ 30 FPS) is approximately 7.5 W, with peak loads of 9–10 W during initialization.

System Assembly for the Project: In this project, the reComputer J1010 unit is mounted on an acrylic base plate with vibration-isolated spacers. The CSI camera is fixed to an adjustable stand at an angle of 20–25° relative to the vehicle direction and at a height of approximately 1.5 m to ensure optimal license-plate capture. The system communicates with a host laptop via Ethernet for SSH access, dataset transfer, and remote model updates.

Overall, the modular construction of the Seeed Studio reComputer J1010 provides mechanical robustness, efficient thermal design, and ease of expansion, making it a reliable platform for continuous real-time AI inference at the edge [Stu23; Mit22].

3.4. Interfaces

The Seeed Studio J101 carrier board exposes multiple connectors for communication, expansion, and debugging. Each interface corresponds to the labeled connector on the PCB as defined in the official datasheet [Stu23].

3.4.1. J1 - J19 Connectors

J10 – 40-Pin Expansion Header (GPIO)

Provides 3.3 V general-purpose I/O pins supporting digital I/O, I^2C, SPI, UART, I^2S, and PWM. Compatible with the Jetson.GPIO Python library for user-space control.

J14 – Serial and Button Header (UART / Debug)

Exposes UART2 (TX/RX) for serial debugging, along with SYS_RESET, PWR_BTN, and FORCE_-RECOVERY pins for power and firmware control. Auto power-on can be disabled by shorting pins 5–6.

J1–J5 – USB Interfaces

- **J1:** USB-C power input (5 V, up to 4 A)
- **J2:** USB-C data/recovery port
- **J4:** USB 3.0 Type-A (5 Gbps)
- **J5:** Dual USB 2.0 Type-A (480 Mbps total, 2 A shared VBUS)

J6 – Gigabit Ethernet Interface

Standard RJ-45 connector supporting 10/100/1000 BASE-T with link (green) and activity (yellow) LEDs. Optional Power-over-Ethernet (PoE) is supported via additional circuitry.

J3 – HDMI Interface

HDMI 2.0 Type-A connector supporting up to 4K @ 60 Hz video output with audio, CEC, and DDC communication.

J11 – M.2 Key E Socket

Provides PCIe x1, USB 2.0, UART, I^2S, and I^2C interfaces for Wi-Fi/Bluetooth modules.

J9 & J13 – Camera (CSI) Interfaces

Two 15-pin MIPI CSI-2 connectors supporting dual 4-lane cameras. Controlled through I^2C lines for power and sensor configuration.

J8 – Fan Connector

4-pin PWM header (5 V, PWM, Tach, GND) for optional active cooling. PWM control is available via GPIO14, and tachometer feedback via GPIO08.

J18 – RTC Coin Cell Battery Holder

Provides backup power for the Real-Time Clock (RTC) using a CR2032 cell.

J19 – SDIO Interface

Dedicated microSD interface supporting 1.8 V/3.3 V operation for SD cards or SDIO peripherals.

Summary of Jetson Nano Interfaces

Table 3.3.: Summary of Jetson Nano Interfaces (reComputer J1010)

Header / Port	**Type / Connector**	**Key Functions**	**Voltage / Speed**	**Remarks**
J1	USB-C	Power Input	5 V ± 0.25 V, 4 A	Main power supply
J2	USB-C	Data / Recovery	USB 2.0	Device mode
J3	HDMI Type-A	Display Output	HDMI 2.0 (4K@60 Hz)	With audio
J4	USB 3.0 Type-A	Host Port	5 Gbps	Camera or SSD
J5	Dual USB 2.0 Type-A	Peripheral Ports	480 Mbps	Keyboard, Mouse
J6	RJ-45 Ethernet	Networking	1 Gbps	PoE optional
J8	4-Pin Fan	Cooling Control	5 V PWM	Optional fan
J9 / J13	15-Pin CSI	Camera Input	4-Lane MIPI	Dual camera support
J10	40-Pin Header	GPIO / I^2C / SPI / UART / I^2S	3.3 V logic	Expansion header
J11	M.2 Key E	PCIe x1, USB2, I^2C	3.3 V	Wireless modules
J14	Button / UART	Power & Debug Control	3.3 V	System interface
J18	Coin Cell Holder	RTC Backup	3 V	CR2032 battery
J19	SDIO Interface	Storage Interface	1.8 V/3.3 V	microSD slot

3.4.2. Pin Configuration

All general-purpose I/O pins on J10 operate at **3.3 V logic levels**. They support configurable modes such as digital I/O, PWM, SPI, UART, I^2C, and I^2S. Over-voltage above 3.3 V can damage the SoC pins. Pin control and configuration are performed through the Jetson.GPIO library or the device tree [Stu23].

3.5. Data Quality

The Jetson Nano offers consistent high-resolution image capture through its dedicated MIPI CSI-2 interface, minimizing latency compared to USB cameras. GPU-accelerated image preprocessing ensures robust frame quality and stability for license plate detection under variable illumination [Mit22].

3.6. Data Quantity

The CSI camera streams up to 30 FPS at 720p or 1080p resolution, producing 2–4 MB per frame. A 64–128 GB microSD card or USB 3.0 SSD provides sufficient capacity for long-duration data recording.

3.7. Specification of Jetson Nano

Table 3.4.: Technical Specifications of NVIDIA Jetson Nano (B01 Revision)

Parameter	**Specification**
GPU	128-core NVIDIA Maxwell architecture
CPU	Quad-core ARM Cortex-A57 MPCore
Memory	4 GB LPDDR4 @ 1600 MHz (25.6 GB/s)
Storage	16 GB eMMC + microSD expansion
Camera Support	Dual CSI-2 (4-lane each)
Networking	Gigabit Ethernet (10/100/1000 Mbps)
Video Decode	Up to 4K @ 60 Hz (HEVC/H.264)
Video Encode	Up to 4K @ 30 Hz
Power Input	5 V DC, up to 4 A (USB-C)
Typical Power Use	5–10 W
Operating System	NVIDIA JetPack 5.x (Ubuntu 20.04 LTS)

3.8. Constraints

- Limited 4 GB RAM restricts execution of multiple large AI models concurrently.
- All GPIO pins operate strictly at 3.3 V logic; higher voltages are not supported.
- No native NVMe slot; high-speed storage achievable via USB 3.0 SSD.
- Continuous GPU inference may require active cooling.
- Stable 5 V ± 5% power supply is required to prevent brown-out resets.

3.9. Dimension of Jetson Nano

- Module Size: 69.6 mm × 45 mm
- Carrier Board Size: 100 mm × 80 mm
- Enclosure Size: 130 mm × 120 mm × 50 mm
- Total Weight: approximately 300 g (including case and heatsink)

3.10. Simple Code for Jetson Nano Reference Carrier Board

The following Python program demonstrates GPIO functionality by blinking an LED connected to pin 12 on the J10 header.

Listing 3.1: LED Blink Test on Jetson Nano

```
import Jetson.GPIO as GPIO
import time

LED_PIN = 12
GPIO.setmode(GPIO.BOARD)
GPIO.setup(LED_PIN, GPIO.OUT, initial=GPIO.LOW)

try:
while True:
GPIO.output(LED_PIN, GPIO.HIGH)
time.sleep(0.5)
GPIO.output(LED_PIN, GPIO.LOW)
time.sleep(0.5)
except KeyboardInterrupt:
GPIO.cleanup()
```

This code verifies GPIO operation and confirms the Jetson Nano's correct hardware configuration under JetPack 5.x.

4. Sensor XY

Introduction

WS:cite books
applications, l

4.1. General

General description
cite books

4.2. Specific Sensor/Actor

cite board

4.3. Specification

- Cite the data sheet
- Extract te information from the data sheet
- Circuit Diagram

4.4. Library

4.4.1. Description

4.4.2. Installation

- data types
- data structure
- data quantity
- data quality

4.4.3. Functions

4.5. Example

In this section, a simple example is described in detail, which demonstrates how the library supports the sensor/actor.

4.5.1. Manual

4.5.2. Inside the Example

4.5.3. Code

4.5.4. Files

4.6. Calibration

cite method

4.7. Simple Code

4.8. Simple Application

4.9. Tests

4.9.1. Simple Function Test

4.9.2. Test all Functions

4.10. Further Readings

5. Actor XY

Introduction

WS:cite books
applications, l

5.1. General

General description
cite books

5.2. Specific Sensor/Actor

cite board

5.3. Specification

- Cite the data sheet
- Extract te information from the data sheet
- Circuit Diagram

5.4. Library

5.4.1. Description

5.4.2. Installation

- data types
- data structure
- data quantity
- data quality

5.4.3. Functions

5.5. Example

In this section, a simple example is described in detail, which demonstrates how the library supports the sensor/actor.

5.5.1. Manual

5.5.2. Inside the Example

5.5.3. Code

5.5.4. Files

5.6. Calibration

cite method

5.7. Simple Code

5.8. Simple Application

5.9. Tests

5.9.1. Simple Function Test

5.9.2. Test all Functions

5.10. Further Readings

Part III.

Domain Knowledge - Machine Learning/Algorithms

6. Algorithm XY

Part IV.

Domain Knowledge - Tools

7. Python Tool XY

8. Hardware Tool XY

Part V.

Methodology

9. KDD XY

Part VI.

Developmenmt

10. Flow Chart Development XY

11. Database XY

12. Data Selection XY

13. Data Transformation XY

14. Data Mining XY

Part VII.

Development to Deployment

15. Machine Learning Pipeline XY

16. Database XY

Part VIII.

Deployment

17. Flow Chart Deployment XY

Part IX.

Monitoring

18. Monitoring XY

Part X.

Verification - Evaluation - Conclusion

19. Evaluation/Verification XY

20. To DoX Y

21. Conclusion XY

Part XI.

Appendix

22. Bill of Material

23. SBOM

requirements.txt

24. Package Example

24.1. Introduction

24.2. Description

24.3. Installation

`pip packageExample`

24.4. Example - Manual

24.5. Example

24.6. Example - Code

24.7. Example - Files

`PackageExample.py`

24.8. Further Reading

References

[Aba+16] M. Abadi et al. "TensorFlow: A System for Large-Scale Machine Learning". In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, pp. 265–283. ISBN: 978-1-931971-33-1. URL: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi.

Part XII.

Hints - Examples for LaTeX - Read and Use

25. Hints

Please, use **`biber.exe`**.

If you want to create a symbol directory, call makeindex:
`makeindex %.nlo -s nomencl.ist`

25.1. Allgemeine mathematische Beschreibung Bézier-Kurve

Bézier curves are formed with the help of Bernstein polynomials. If at least two points and two tangents are known, control points can be determined with which a control polygon is formed. The course of the curve is oriented to this control polygon. The number of control points depends on the degree of the Bézier curve. A Bézier curve of degree n has n+1 control points. The Bézier curve is calculated using the De Casteljau algorithm.

Definition. In the interval [0; 1] the **Bernstein polynomial of nth degree** is defined by:

$$b_{i,n}(\lambda) = \binom{n}{i}(1-\lambda)^{n-i}\lambda^i, \quad \lambda \in [0,1], \quad i = 0, \ldots, n \tag{25.1}$$

Definition. The binomial coefficient is defined by:

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}, \quad i = 0, \ldots, n \tag{25.2}$$

Here the Bernstein polynomials $b_{i,n}$ form a basis of the vector space $\mathbb{P}^n(I)$ of polynomials of degree at most n over I. Thus every polynomial of degree at most n can be written uniquely as a linear combination. [Far02]

Beispiel. Determination of Bernstein polynomials for $n = 3$ holds:

$$\begin{aligned}
b_{3,0}(\lambda) &= \binom{3}{0}(1-\lambda)^{3-0}\lambda^0 &&= (1-\lambda)^3 \\
b_{3,1}(\lambda) &= \binom{3}{1}(1-\lambda)^{3-1}\lambda^1 &&= 3\lambda(1-\lambda)^2 \\
b_{3,2}(\lambda) &= \binom{3}{2}(1-\lambda)^{3-2}\lambda^2 &&= 3\lambda^2(1-\lambda) \\
b_{3,3}(\lambda) &= \binom{3}{3}(1-\lambda)^{3-3}\lambda^3 &&= \lambda^3
\end{aligned}$$

$b_{3,i}(\lambda)$ with $i = 0, 1, 2, 3$ are the cubic Bernstein polynomials of degree 3.

Definition. Given are the points

$$Q_i = \begin{pmatrix} x_i \\ y_i \end{pmatrix} \quad \text{mit} \quad Q_i \in \mathbb{R}^2, \quad i = 0, 1, \ldots, n$$

A **Bézier curve** is then defined by

$$C(\lambda) = \sum_{i=0}^{n} Q_i \cdot b_{i,n}(\lambda) \tag{25.3}$$

The points $Q_i, i = 0, \ldots, n$ are called **control points**.

The control points of a Bézier curve form the so-called control polygon.

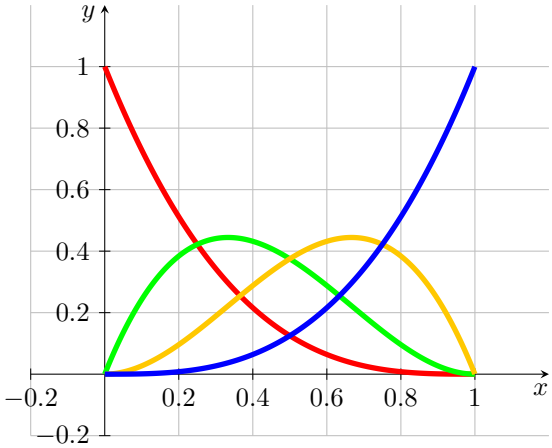


Figure 25.1.: Bernstein polynomial of degree 3

Bemerkung. Let the starting point P_0 and the end point P_1, as well as the tangents $\vec{t}_0$ and $\vec{t}_1$ be given. The tangents are not necessarily normalised.
The control points Q_0, Q_1, Q_2 and Q_3 of the associated Bézier curve are given by the following equations:

$$Q_0 = P_0, \qquad Q_1 = P_0 + \lambda_0 \vec{t}_0, \qquad Q_2 = P_1 - \lambda_1 \vec{t}_n, \qquad Q_3 = P_1 \tag{25.4}$$

[Jak+10]

Beispiel. Given are

$$P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad \vec{t}_0 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \text{und} \quad P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}, \quad \vec{t}_1 = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Then, according to theorem **??** for control points of the associated Bézier curve results:

$$Q_0 = P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \qquad Q_1 = P_0 + \vec{t}_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \end{pmatrix},$$

$$Q_2 = P_1 - \vec{t}_n = \begin{pmatrix} 6 \\ 1 \end{pmatrix} - \begin{pmatrix} 1 \\ -2 \end{pmatrix} = \begin{pmatrix} 5 \\ 3 \end{pmatrix}, \qquad Q_3 = P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}$$

The Bézier curve and its control points are shown in the figure **??**.

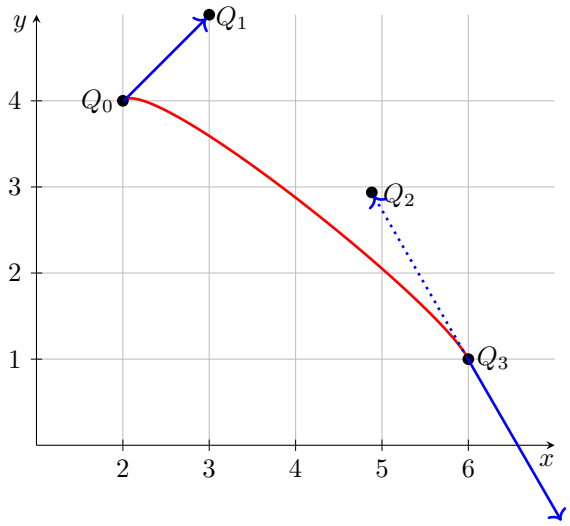


Figure 25.2.: Bézier curve for example 25.1

26. Verrundung mit einem Kreisbogen

26.1. Gleichungen

Blending with an arc is a simple variant of smoothing corners. This variant enables the processing and the generation of files according to DIN 66025. For smoothing, three points P_0 and S and P_1 are always considered, thus a symmetric Hermite problem results. According to theorem **??** it is assumed to be in the standard form $HP(L, \alpha)$.

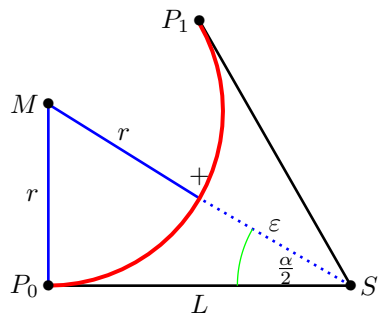


Figure 26.1.: Smoothing a corner with the help of an arc - triangle

The figure **??** represents the situation. The points P_0, S and P_1 are given. The included angle $\alpha = \angle(P_0; S; P_1)$ as well as the distance $L = \|\|S - P_0\|\| = \|\|P_1 - S\|\|$ are shown in the graph. The red arc is the desired result. The distance of its centre M to S is then $r + \varepsilon$, where r is the radius of the arc and ε is the given tolerance. Thus we obtain a right triangle P_0SM whose edge lengths are L, $r + \varepsilon$ and r.

According to the definition of the sine and the tangent, it follows:

$$\sin\left(\frac{\alpha}{2}\right) = \frac{L}{r+\varepsilon} \qquad \text{und} \qquad \tan\left(\frac{\alpha}{2}\right) = \frac{L}{r}$$

$$\Leftrightarrow \quad \varepsilon = \frac{L}{\sin\left(\frac{\alpha}{2}\right)} - r \qquad \text{und} \qquad r = \cot\left(\frac{\alpha}{2}\right) L$$

The two equations can be combined to give the following conditions:

$$\varepsilon = L \cdot \frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} \qquad \text{bzw.} \qquad L = \varepsilon \cdot \frac{\sin\left(\frac{\alpha}{2}\right)}{1 - \cos\left(\frac{\alpha}{2}\right)}$$

This gives the equation for the radius:

$$r = L \cdot \cot\left(\frac{\alpha}{2}\right) = L \cdot \sqrt{\frac{1 - \cos(\alpha)}{1 + \cos(\alpha}} = L \cdot \frac{\sin(\alpha)}{1 + \cos(\alpha)} = L \cdot \frac{P_{1,x}}{P_{1,y}}$$

The factor for converting L and ε can be simplified using trigonometric transformations.

$$\frac{1-\cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{1-\sqrt{\frac{1-\cos(\alpha)}{2}}}{\sqrt{\frac{1+\cos(\alpha)}{2}}} = \frac{\sqrt{2}-\sqrt{1-\cos(\alpha)}}{\sqrt{1+\cos(\alpha)}} = \frac{\sqrt{1+\cos(\alpha)}-\sin(\alpha)}{1+\cos(\alpha)}$$

By comparing this with the symmetric Hermite problem in standard form, the following notation is then obtained:

$$\frac{1-\cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{\sqrt{P_{1,x}}-P_{1,y}}{P_{1,x}}$$

The previous considerations are now summarised in the following sentence.

Satz. Let a symmetric Hermite problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ be given. Without restriction of generality, it is in the standard form

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}L+L\cdot\cos(\alpha)\\L\cdot\sin(\alpha)\end{pmatrix}, \begin{pmatrix}\cos(\alpha)\\\sin(\alpha)\end{pmatrix}, \begin{pmatrix}L\\0\end{pmatrix}, L\right\}$$

with $L \in \mathbb{R}^{>0}$ uad $\alpha \in (-\pi;\, \pi]$.

Then an arc can be found that connects the points P_0 and P_1, has the same tangent directions at the two points.

For the circular arcs applies:

$$r = L\cdot\left|\frac{P_{1,x}}{P_{1,y}}\right|; \quad \phi_0 = -\operatorname{sign}(\alpha)\cdot\frac{\pi}{2}; \quad \phi_1-\phi_0 = \operatorname{sign}(\alpha)\cdot\pi-\alpha = \beta;$$

$$M = P_0 + \operatorname{sign}(\alpha)\cdot r\cdot\vec{t}_0^{\perp} = \operatorname{sign}(\alpha)\cdot r\cdot\begin{pmatrix}0\\1\end{pmatrix}$$

From the specification of the distance L, the maximum error can be calculated, as shown in theorem **??**. According to the derivation, it is also possible to specify the maximum error ε and determine the maximum distance L from it.

Satz. Let a symmetric Hermite problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ be given. Without restriction of generality, it is in the standard form

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}L+L\cdot\cos(\alpha)\\L\cdot\sin(\alpha)\end{pmatrix}, \begin{pmatrix}\cos(\alpha)\\\sin(\alpha)\end{pmatrix}, \begin{pmatrix}L\\0\end{pmatrix}, L\right\}$$

with $L \in \mathbb{R}^{>0}$ and $\alpha \in (-\pi;\, \pi]$.

a) Let the maximum error ε be given. The maximum distance L for which an arc exists according to the theorem **??** that takes the error into account is given by:

$$L(\varepsilon,\alpha) = \varepsilon\cdot\frac{P_{1,x}}{\sqrt{P_{1,x}}-P_{1,y}} = \varepsilon\cdot\frac{L+L\cdot\cos(\alpha)}{\sqrt{L+L\cdot\cos(\alpha)}-L+L\cdot\cos(\alpha)}$$

b) When the distance L is specified, the following maximum error results:

$$\varepsilon(L,\alpha) = L\cdot\frac{\sqrt{P_{1,x}}-P_{1,y}}{P_{1,x}} = L\cdot\frac{\sqrt{L+L\cdot\cos(\alpha)}-L+L\cdot\cos(\alpha)}{L+L\cdot\cos(\alpha)}$$

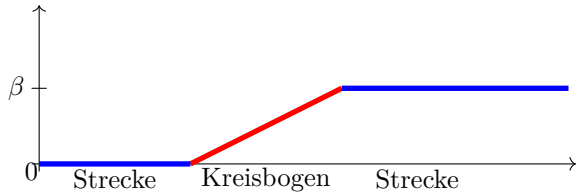


Figure 26.2.: Angular change with blending arc

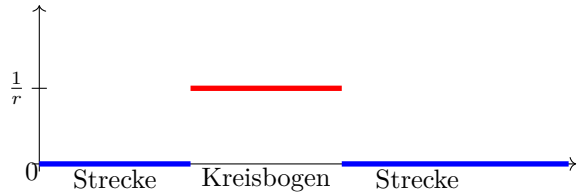


Figure 26.3.: Curvature progression for a blending arc

These considerations result in limitations for the use of this strategy, which are summarised in the following comment.

Bemerkung. The following conditions for applying the strategy of a circle must be fulfilled:

a)
$$P_0 \neq P_1$$

b)
$$\vec{t_0} = -\vec{t_1} \quad \Leftrightarrow \quad \alpha = 0$$

c)
$$\vec{t_0} = \vec{t_1} \quad \Leftrightarrow \quad \alpha = \pi$$

26.2. Bewertung

Rounding by means of a circular arc leads to a continuous course of the angle change. The figure **??** illustrates this.

Due to the rounding with a circular arc, the curvature of the curve is not continuous. The figure **??** shows that the curvature is constant in the range of distances 0 and on the circular arc with the value $\frac{1}{r}$, where r is the radius of the circular arc used. Due to the formula $a = \frac{v^2}{r}$ for a movement on a circular arc, the acceleration course exhibits continuity jumps at the transitions for a constant path velocity.

Depending on the angle change β at the corner S and the tolerance ε, the maximum length of the shortening of the lines can be calculated. The figure **??** shows the corresponding diagram, where the tolerance ε is set to the value 1.

The area provided can also be specified. Depending on the distance L from the corner point S, the deviation ε can be calculated. The figure **??** represents the function as a function of L and the angle α. The figures **??** and **??** show the curves of the length as a function of the angle change for a fixed tolerance and the error as a function of the angle change for a given length. If the angle change is minimal, then

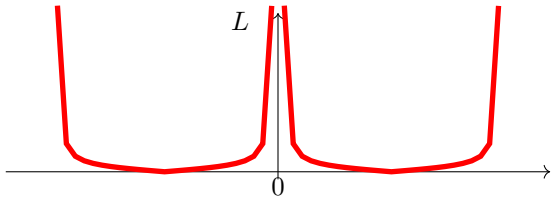


Figure 26.4.: Maximum range for a blending arc of a circle - $L(\varepsilon = const, \alpha)$

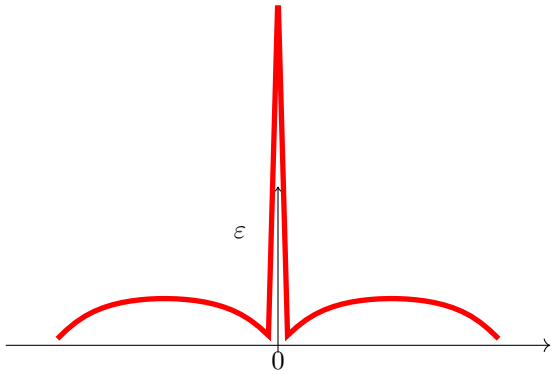


Figure 26.5.: Deviation when specifying the distance L when rounding with a circular arc

the error or length L is extreme. In this case, rounding by means of an arc is not possible. In order to develop a sensible strategy, a minimum angle change must be specified from which a blemnding arc is permitted. Likewise, a maximum angle change must also be defined. For an angular change of $\pm\pi$ is a bend. In this case, a blending is also not possible.

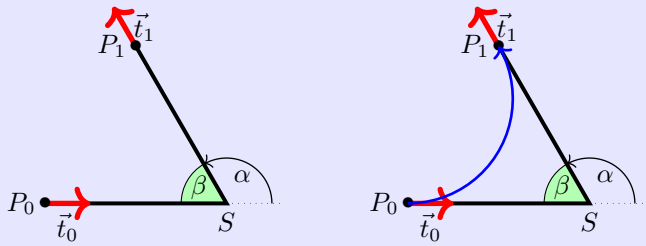
26.3. Examples

Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{2}{3}\pi\right) \\ 6.0 \cdot \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{2}{3}\pi\right) \\ \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = \frac{2}{3}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix} 0 \\ 3,4641 \end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{2}{3}\pi$$

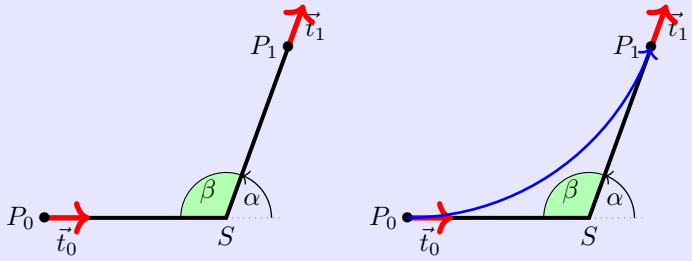


Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{7}{18}\pi\right) \\ 6.0 \cdot \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{7}{18}\pi\right) \\ \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = \frac{7}{18}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix} 0 \\ 8,5689 \end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{7}{18}\pi$$

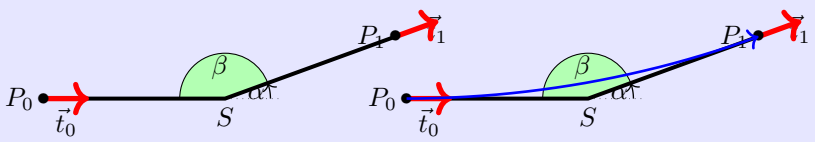


Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{1}{9}\pi\right) \\ 6.0 \cdot \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{1}{9}\pi\right) \\ \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = \frac{1}{9}\pi$.
For the blending arc follows:

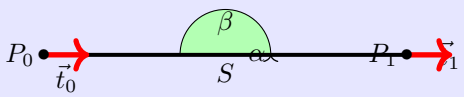
$$M = \begin{pmatrix} 0 \\ 34,0277 \end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{1}{9}\pi$$



Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 12.0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = 0$.
There is no blending arc here.

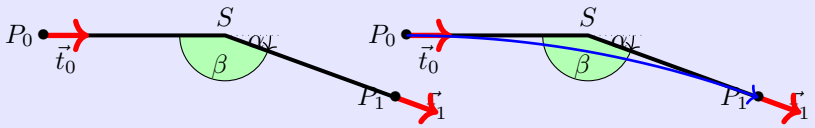


Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{1}{9}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{1}{9}\pi\right) \\ \sin\left(-\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = -\frac{1}{9}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix} 0 \\ -34,0277 \end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = \frac{1}{9}\pi$$

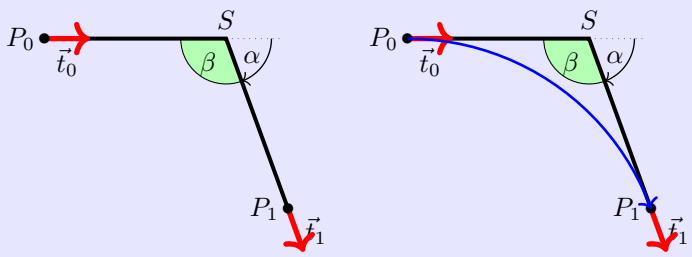


Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{7}{18}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{7}{18}\pi\right) \\ \sin\left(-\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = -\frac{7}{18}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix} 0 \\ -8,5689 \end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{7}{18}\pi$$

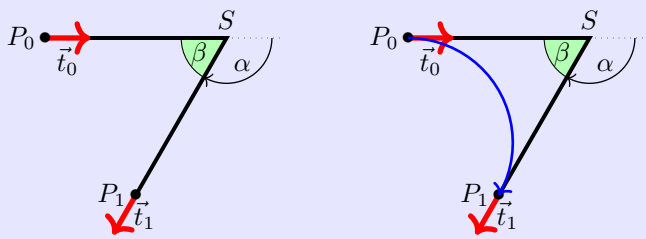


Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{2}{3}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{2}{3}\pi\right) \\ \sin\left(-\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = -\frac{2}{3}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix} 0 \\ -3,4641 \end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{2}{3}\pi$$



27. Maple files

[Wat17c; Wat17d; Wat17b; Wat17e; Wat17a; Wat17f]

27.1. Geometries with Maple

The algorithms presented can be well represented using geometries. Two aspects can be investigated. On the one hand, the effort for an implementation, the computational accuracy and the stability of an algorithm are interesting; on the other hand, the methods are to be evaluated and compared with regard to their usefulness. For this purpose, modules for the formula manipulation system Maple [Wat17c] were developed.
Maple offers the environment to implement algorithms easily and quickly. Furthermore, graphical representation is possible with simple means. However, a simple workbook of Maple is not designed for very large software projects. Here, one must resort to the possibility of creating and using libraries. On the one hand, Maple offers the possibility of creating one's own libraries, so-called modules. In this way, one remains within the environment and syntax of Maple. This path will be pursued further here. Another possibility is the use of libraries created by means of a higher programming language, e.g. C++, the so-called DLLs.
In the following, the creation and use of a test environment with the help of modules is described. First, the geometry elements that are stored in various modules and their use are described. The structure and use of a module in Maple is then explained so that own extensions and additions are possible.

27.2. Geometry elements used

This is a test environment for the algorithms presented, only geometries that have been described are also used.

- points (`MPoint`)
- lines (`MLine`)
- arcs (`MArc`)
- Bézier curves (`Bezier`)
- polygon courses (`MPolygon`)
- Geometry List (`MGeoList`)
- Hermite problems (`MHermiteProblem`)
- Symmetric Hermite Problems (`MHermiteProblemSym`)

For each geometry element a corresponding module has been created, the name of which is given in the list above. The list of which functions are available is presented in another section.
When implementing such a project, it quickly becomes clear that when using several geometry elements, a clear data structure and well-defined access to the data is essential. For example, when representing straight lines, the decision must be made whether the representation should be point-directional or by means of two points. The choice is generally made depending on the application. Here, the path is taken that, due to a well-defined access to the data, the use of both representations is possible.

27.3. Building the data structure

The idea is to use a data structure that is as uniform and simple as possible. Maple does offer the possibility to define objects. However, it is difficult to use within a procedural environment. Therefore, all data is basically represented as lists. The first list element always contains an identifier of the element **??**. This is followed by the data, which in turn can be geometry elements. It should be noted that direct access to the data cannot be prevented; here the user is responsible.

Access to the data should be exclusively via procedures of a module. The following is an example of the data structure for points:

```
[MVPOINT,[x,y]]
```

The creation of a point then takes place via a procedure `New`:

```
NeuerPunkt := MPoint:-New(10,15);
```

When called up in the test file, the following is then output:

```
TestP0 := ["Point",[10,15]]
```

These data structures are used for the calculation of geometries. They are used in functions or procedures. Unlike variables, the data structures and procedures are defined globally and not locally. Under the command `export` the procedures are declared at the beginning of the module and can thus also be used outside the module. If, for example, a data structure from `MPoint` is to be used in another module or outside the modules, it is called as follows:

```
P0 := MPoint:-New(x,y);
```

In addition to the modules for the geometries, there is a module (MConstant) for saving constants and names. There, for `MVPOINT`, for example. „Point “ or e.g. a constant for the comparison to zero for real numbers.
Finally there is the test file, which is not a module, in which the modules are loaded, called and tested in their function. More about this file later in „1.4 Maple test file “.

27.4. Structure of a module

The following section deals with the purpose of modules and their structure.

The project is about capturing the above geometries in a data structure and representing them in Maple. However, the calculations and formulas that have to be used are a hindrance to the final representation. Modules are very well suited for summarising and hiding the calculations that take place in functions. This way, the important functions can be accessed in Maple without seeing what is in them.

Example:
The function `Angle` from the module `MPoint`: Function to calculate an angle between location vector and x-axis:

```
Angle := proc(P)
   local alpha, x, y;
   x := GetX(P);
   y := GetY(P);
   alpha := 0;
   if abs(x) < 0.00001
      then
         alpha := 3*Pi/2;
```

```
      else
         alpha := Pi/2;
      end if;
   else
      alpha := arctan(y/x);
      if x < 0
      then
         alpha := alpha + Pi;
      else
         if y < 0
         then
            alpha := alpha + 2*Pi;
         end if;
      end if;
   end if;
   return factor(alpha);
end proc;
```

This function is long, but it only represents a small part of the programme for the representation in question. That is why it is in the module and can thus be called externally with a single command:

```
Winkel := MPoint:-Angle(P)
```

The following section describes the structure of a module. Since Maple offers a wide range of possibilities, a restriction is made. Only the structure of the modules used is described, here on the basis of the moduleMPoint. For more detailed possibilities, please refer to the Maple manual [Wat17c].

<u>structure:</u>

The module must first be started. This is done with the name (here always a capital M and the geometry) of the module and the following command:

```
MPoint := module()
```

Then, similar to procedures, the variables and functions must be defined. You can declare them either locally or globally. If the variables or procedures are only used and changed in the module, the declaration is made with the command `local` as follows:

```
local Variable names, with, comma, separated;
```

If the variables or procedures are also to be usable outside the module, this is defined with `export`:

```
export Function names, with, comma, separated;
```

This is followed by the specification of the options:
`option package;`,
the description of the module:
`description “Self-selected module description, e.g. module for points “`;
and the initialisation of the module:

```
ModuleLoad := proc
  MVPOINT:=MConstant:-GetPoint();
  print("Module MPoint is loaded");
end proc;
```

From here on, programming is done as usual in Maple. The previously defined procedure and variable names are used and programming is done with commands that are also used outside of a module in Maple. It is important to know that with modules, each function can be called constantly, so the order of the functions is irrelevant.

To finally end the module, use the following command:

```
end module;
```

27.4.1. General structure of a module

In summary, the modules have the following structure:

```
Modulname := module()

   local Names, with, comma, separated;

   export Names, with, comma, separated;

   global Names, with, comma, separated;[1]

   option package;

   description „ Self-selected module description“;


   ModuleLoad := proc()[2]
      MVPOINT:=MConstant:-GetPoint();
      print("Modul MPoint is loaded");
   end proc;

 Procedure1 := proc (Passing parameters)
      inhalt;
   end proc;

   Procedure2 := proc (Passing parameters)
      content;
   end proc;


   ...

end module;
```

[1] Possible, but not used in the modules
[2] Example `MPoint`

27.4.2. Saving a module

The management of modules is done automatically by Maple. However, since the modules are passed on and have to be edited individually, some settings have to be made. Therefore, the following prefix is used for each Maple file of the project:

```
restart;
with(LibraryTools);
lib := "C:/FH/Tools/Maple/MyLibs/Blending.mla";
march('open', lib);
ThisModule := 'MArc';
```

The first line initialises the system. The next 3 lines enable working with archives. In the second line, the tools are loaded so that the variable `lib` can be occupied. All modules are stored in an archive; in this example it is the file `Blending.mla` in the directory `C:/FH/Tools/Maple/MyLibs/`. The command `ThisModule := 'MArc';` contains the name of the current module.

A module is then saved using the command `savelib('ModuleName')`. A file `ModuleName.mla` is then created. Depending on the configuration of Maple, the set path where the file is automatically created may not be writable. Then you can configure the system so that the mla file is stored in the current directory or in a directory of your choice.

If the above prefix is used for a Maple file, all that is required to save the module is
`savelib(ThisModule, lib);`

27.4.3. Verwendung eines Moduls

A module that has been saved can now be used in other Maple worksheets. To do this, the command
`with(ModuleName)`
is used. If you have used a special path, Maple cannot find the file `ModuleName.mla`. Then the path must be made known, e.g.:
`savelibname := "c:/Maple/MyLibs";", savelibname;`
Now the procedures of the module can be accessed. An overview of all exported procedures is provided by the command
`Describe(ModuleName)`
is displayed. A list of the names including the names of the transfer parameters is displayed. If a procedure is equipped with the field `description`, this text is also displayed.

27.4.4. Creating a Maple module

Creating your own module is done quickly if you use the framework above. However, one should make some considerations beforehand and maintain a standard.

Before creating an own module, the data model and the corresponding procedures should be worked out. A programme flow chart is useful here. The central task of the module is usually quickly determined. In addition, however, the following rules should be observed.

Rule 1 Basically, both the module and each procedure receive a description. This is not limited to the optional argument `description`, but is placed in front of each procedure. The description basically contains the description of the task. The prerequisite is also mentioned or an error handling is described. Then follows the description of all input parameters, their function and their data structure. The return value is then described.

rule 2 Each module receives a procedure `Version()`, which returns the current version number.

rule 3 Data is not global. To access data, procedures `Set*` and `Get*` are provided. These procedures are also used within the procedures of a module.

rule 4 At least one test function is written for each procedure. The test function can be used to illustrate the use and any special features.

27.5. Functions of the modules

In the following, the individual modules are listed. The data structure of each module and all functions with associated tasks are listed.

27.5.1. Module MPoint

MPoint is a module for points and works with a data structure and with procedures/functions. The data structure New for a point is a list, which is structured as follows:

```
[MVPOINT, [x,y]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is "Point" and the elements are the x and y coordinates for a point. No distinction is made here between point and vector. A point can also be seen as a vector from the coordinate origin to the point.
Via the Get functions GetX and GetY one can capture the coordinates of the point and use them in other functions. The other functions can then be used to calculate with the points/vectors.

MPoint		
E	**New**	Data structure for a point
E	**GetX**	Reading the x-coordinate
E	**GetY**	reading the y-coordinate
E	**Angle**	calculating the angle between the x-axis and the point
E	**Add**	Calculates a linear combination of two points/vectors
E	**Sub**	Calculates the difference of two points/vectors
E	**Cos**	Calculates the cosine between two vectors
E	**Sin**	Calculates the sine between two vectors
E	**Scale**	Scales a vector with a factor
E	**Perp**	Calculates a vector that is orthogonal to the given vector
L	**IllustrateXY**	Plot function to illustrate a blue point
E	**Illustrate**	Plot of a blue point
E	**Plot**	Plot of a green point
E	**Plot2D**	Plot of a point with own options
E	**Length**	Calculates the distance of the point to the coordinate origin
E	**Uniform**	Normalises the vector to length 1
E	**LinetoVector**	Calculates vector from a distance
E	**Distance**	Calculates the distance between two points

27.5.2. Module MLine

MLine is a module for lines and works with a data structure and with procedures/functions. The data structure New for a line is a list which is structured as follows:

```
[MVLINE, [P0,P1]]
```

Its first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „Line“ and the elements are the start and end points for a line.
The data structure NewPointVerctor is also a data structure for a line, but here the line is defined by a start point and a direction vector.

The Get functions StartPoint and EndPoint can be used to capture the start and end points of the route and use them in other functions. The other functions can then be used to calculate with the routes.

MLine		
E	**New**	Data structure for a line (two-point form)
E	**NewPointVector**	creation of a route (point-direction form)
E	**StartPoint**	reading the starting point
E	**EndPoint**	reading the end point
E	**Position**	Calculate a point that lies on the line
E	**Plot2D**	Plot a part of the route starting from the starting point
E	**Plot2DTangent**	Plot of a point with tangent
E	**Plot2DTangentArrow**	Plot of a point with tangent (as arrow)
E	**LineLine**	Calculation of the intersection of two straight lines.
E	**AngleLine**	Calculation of the angle between two straight lines

27.5.3. Module MArc

MArc is a module for circular arcs and works with a data structure and with procedures/functions. The data structure New for an arc is a list that is structured as follows:

```
[MVARC, [mx,my,r,phi,alpha]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is "Arc" and the elements are the x- and y-coordinate for the centre, the radius, the start angle and the angle change for an arc.

The Get functions GetM, GetMX, GetMY, GetR, GetPhi, and GetAlpha can be used to collect the data for an arc and use it in other functions. The other functions can then be used to calculate with the arcs.

MArc		
E	**New**	Data structure for an arc
E	**GetMX**	Reading the x-coordinate of the centre point.
E	**GetMY**	read the y-coordinate of the centre point
E	**GetR**	read the radius
E	**GetPhi**	reading the start angle
E	**GetAlpha**	Reading the change of angle
E	**GetM**	reading the centre point
E	**Position**	Calculating a point on the arc
E	**Plot2D**	Plot a part of the arc starting from the start angle
E	**Blend**	calculating an arc from a symmetric Hermite problem

27.5.4. module MBezier

The New data structure for a polygon is a list constructed as follows:

```
[MVBEZIER, [PointList]]
```

Within the module MBezier exist the procedures listed in the following table.

MBezier		
E	New	Manual input of control points for a Bézier curve
E	Version	Output of the verions
E	BlendCurvature	Determination of control points from symmetric Hermite problem
E	BlendCurvatureEpsilon	determination of control points from symmetric Hermite problem with given error
E	Position	position on the Bézier curve
E	GetTheta	Reading the angle
E	GetEpsilon	reading the tolerance
E	GetControlPoint	Read the control points
E	Plot2D	Read the Bézier curve
E	PlotControlPoints	representation of all control points

27.5.5. Module MPolygon

MPolygon is a module for polygons and works with a data structure and with procedures/functions. The data structure New for a polygon is a list that is structured as follows:

```
[MVPOLYGON, [PointList]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is "Polygon" and the elements are any number of points. Using the Get functions GetPoint and GetN you can get the number of points and the points themselves and use them in other functions. The other functions can then be used to calculate with the points or the polygon course.

MPolygon		
E	New	Data structure for a list of points
E	GetPoint	Reading the ith point from the point list
E	GetN	Determine the number of points in the point list
E	Length	Determination of the Euclidean length of the polygon
E	Position	Calculation of a point on the polygon course
E	Tangents	calculation of the tangent
L	Plot2DAll	Plot list of all points
E	Plot2D	Plot of all points as polygonal plots.
E	Plot2DTangent	representation of the polygon course with tangent
E	PlotPoints	representation of all points

27.5.6. module MGeoList

MeoList is a module for a geometry list and works with a data structure and with procedures/functions. The data structure New for a geometry list is a list that is structured as follows:

```
[MVGEOLIST, []]
```

Its first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „GeoList“ and the elements are any number of individual geometries.
Using the Get functions GeoGeo and GetN, the i-th element of the list and the number of elements in the list can be captured and used in other functions. The other functions can then be used to calculate with the geometries or the list. In the functions Length, Plot2DAll, Plot2D and Position the functions are called in themselves. This is possible because the functions work independently of each other.

MGeoList		
E	New	Data structure for a geometry list
E	Append	Append a geometry element to the data structure
E	Prepend	Inserting a geometry element as the first element of the list
E	Replace	Replace a geometry element with another one.
E	GetN	Determine the number of geometry elements in the list
E	GeoGeo	Reading the i-th geometry element
E	Length	Calculate the Euclidean length of the geometry elements
E	Position	Calculates a point on the geometry
L	Plot2DAll	Plot function for plotting the geometry elements
E	Plot2D	Plot a part of the geometry list starting from the first element

27.5.7. Module MHermiteProblem

MHermiteProblem is a module for Hermite problems and works with a data structure and with procedures/functions. The data structure New for a route is a list, which is structured as follows:

```
[MVHERMITEPROBLEM, [P0,T0n,P1,T1n]]
```

Your first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „HermiteProb“ and the elements are two points with associated tangents (lines).

Using the Get functions StartPoint, EndPoint, StartTangent and EndTangent, the points and their tangents can be captured and used in other functions. The other functions can then be used to calculate with the data for the Hermite problem.

MHermiteProblem		
E	New	Data structure for a Hermite problem
E	StartPoint	reading the start point
E	EndPoint	Reading the end point command
E	StartTangent	reading the start tangent
E	EndTangent	Reading the end tangent
E	Plot2D	Plotting the Hermite problem

27.5.8. Module MHermiteProblemSym

MHermiteProblemSym is a module for symmetric Hermite problems and works with a data structure and with procedures/functions. The data structure New for a symmetric Hermite problem is a list built as follows:

```
[MVHERMITEPROBLEMSYM, [P0,T0n,P1,T1n,S,L]]
```

Your first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „SymHermiteProb“ and the elements are two points with associated tangents, their intersection, and the distance of the points to the intersection.

Via the Get functions StartPoint, EndPoint, StartTangent, EndTangent, CrossPoint and ParameterL one can acquire the data and use it in other functions. The other functions can then be used to calculate with the data for the symmetric Hermite problem.

MHermiteProblemSym		
E	New	Data structure for a symmetric Hermite problem
E	StartPoint	reading the start point
E	EndPoint	Reading the end point command
E	StartTangent	reading the start tangent
E	EndTangent	Reading the end tangent
E	ParameterL	reading of the distance
E	CrossPoint	reading of the intersection point
E	Plot2D	Plotting of the symmetrical Hermite problem
E	Create	creation of a Sym. Hermite problem by 3 points
E	BlendArc	rounding of the corner point by an arc

27.5.9. Module MConstant

MConstant is a module for storing fixed constants and names to identify data structures. In the locally declared functions, starting with CV, the constants for declaring the different geometries are defined. These are names for recognising the geometry elements in the test file. In the globally declared functions, starting with Get, the names for identifying the geometry elements are returned.

MConstant		
L	NULLEPS	constant to compare to zero
L	CVPOINT	constant for geometry elements: Point
L	CVLINE	constant for geometry elements: Line
L	CVARC	constant for geometry elements: Arc
L	CVPOLYGON	constant for geometry elements: polygon
L	CVGEOLIST	constant for geometry elements: GeoList
L	CVHERMITEPROBLEM	constant for geometry elements: HermiteProb
L	CVHERMITEPROBLEMSYMMETRIC	Const. for geometry elements: SymHermiteProb
L	CVBIARC	Const. for geometry elements: Biarc
E	GetNullEps	return of the zero comparison command.
E	GetPoint	identifier for points
E	GetLine	Identifier for lines
E	GetArc	identifier for circular arcs
E	GetPolygon	identifier for polygons
E	GetGeoList	Identifier for geometry lists
E	GetHermiteProblemSymmetric	Identifier for symmetric Hermite problems
E	GetHermiteProblem	ID for Hermite problems
E	GetBiarc	identifier for Biarcs

27.5.10. Module MGeneralMath

MGeneralMath is a module for general mathematical functions and works with the data structures and procedures.

MeneralMath		
E	MPoint	Data structure for a point
E	MPointX	Reading of the x-coordinate for a point
E	MPointY	reading the y-coordinate for one point
L	MPointIllustrateXY	plot structure for a blue point
E	MPointIllustrate	plot structure for a blue point
E	MPointPlot	Illustration of a green point
E	MLine	Data structure for a line
E	MLineStartPoint	Reading of the start point for a draw frame
E	MLineEndPoint	Reading the end point for a line
E	MPointOnLine	Calculation of a point on the line
E	MLinePlot2D	Plot the part of a line starting at the starting point
. E	MLineLine	calculating the intersection of two linesline

27.5.11. Module Biarc

Data Structure

Requirements and specifications:

The Biarc is to be used to solve a Hermite problem. A Hermite problem is defined as follows:

Given two points P_0 and P_1 with associated normalised tangents $\vec{t_0}$ and $\vec{t_1}$. The biarc must connect the points such that the tangents of the start and end points of the biarc coincide with the tangents of the Hermites problem.

Data structure:

The data structure New for a biarc must contain two arcs.

So the data structure for one arc is needed: MArc:-New.

This in turn contains the coordinates for the centre, the radius, the start angle and the angle change.

27.5.12. Module MBiarc

MBiarc is a module for Biarcs and works with a data structure and with procedures/functions. The data structure New for a Biarc is a list which is structured as follows:

```
[MVBIARC, [Arc0, Arc1]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is „Biarc“ and the elements are two arcs.

Using the Get functions GetArc0 and GetArc1 one can capture the two arcs for the biarc. The functions listed below can be used to calculate the data for the biarc.

MBiarc		
E	New	Data structure for a biarc
E	GetArc0	Reading of the first arc
E	GetArc1	Reading the second arc
E	Circle	calculating the circle K_J from the Hermite problem data.
E	Plot2DCircle	plot of the circle K_J
E	angle	Calculate the angle from the centre of the circle to points on the circle
E	Plot2D	Plot of the biarc
E	ConnectionPoint	Calculation of the connection point J (Equal Chord)
E	TangentTj	Calculation of the tangent to J
E	Tangent Biarc	rotation of Tj for the biarc.
E	BiarcCenter	Calculate the centres of the arcs of the biarc
E	Biarc	calculate the centre of the biarc.
E	Position	Determine a point on the biarc
E	Blend	Calculate the biarc only from the Hermite problem

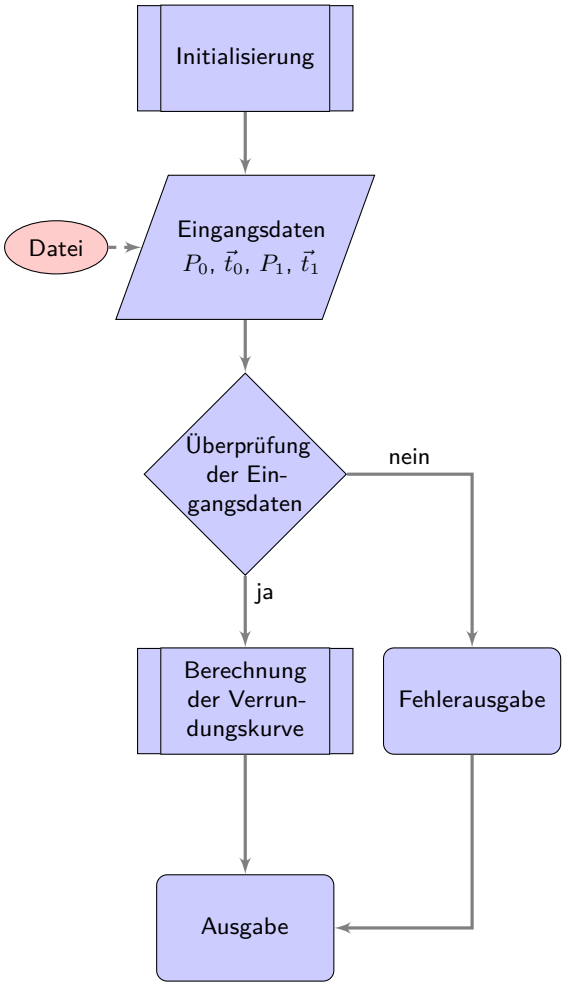


Figure 27.1.: Programme flow chart „Corner rounding“

27.6. Programme Flowchart

27.6.1. Overall Flow

27.6.2. Verrundung der Kurve

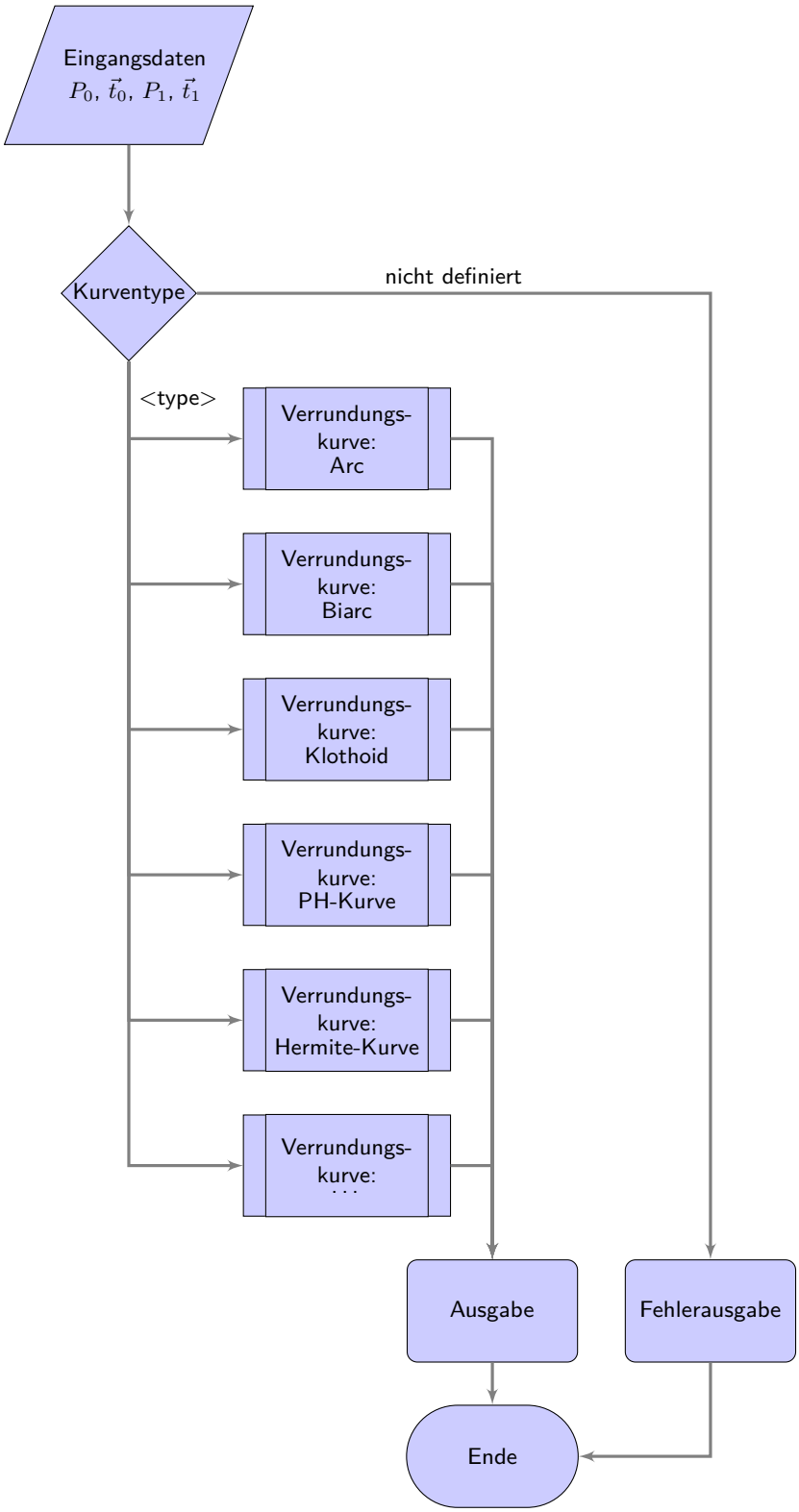


Figure 27.2.: Programme flowchart „Selection of the rounding strategies“

28. Representation of Python Programs

It is possible to integrate a part into the document, see 28.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document.
It may also be useful to integrate program lines: `redLED = pyb.LED(1)`.

Attention: The integration of programs with the help of images is pointless.

Listing 28.1.: The program “Hello World” in Python for microcontroller boards is inserted from the file Blink.py.

```
%%
%% This is file `rename-to-empty-base.tex',
%% generated with the docstrip utility.
%%
%% The original source files were:
%%
%% fileerr.dtx  (with options: `return')
%%
%% This is a generated file.
%%
%% The source is maintained by the LaTeX Project team and bug
%% reports for it can be opened at https://latex-project.org/bugs/
%% (but please observe conditions on bug reports sent to that address!)
%%
%%
%% Copyright (C) 1993-2024
%% The LaTeX Project and any individual authors listed elsewhere
%% in this file.
%%
%% This file was generated from file(s) of the Standard LaTeX `Tools Bundle'.
%% ---------------------------------------------------------------------------
%%
%% It may be distributed and/or modified under the
%% conditions of the LaTeX Project Public License, either version 1.3c
%% of this license or (at your option) any later version.
%% The latest version of this license is in
%%    https://www.latex-project.org/lppl.txt
%% and version 1.3c or later is part of all distributions of LaTeX
%% version 2005/12/01 or later.
%%
%% This file may only be distributed together with a copy of the LaTeX
%% `Tools Bundle'. You may however distribute the LaTeX `Tools Bundle'
%% without such generated files.
%%
%% The list of all files belonging to the LaTeX `Tools Bundle' is
%% given in the file `manifest.txt'.
%%
 \message{File ignored}
\endinput
%%
%% End of file `rename-to-empty-base.tex'.
```

../../Code/HelloWorld/Blink.py

29. Representation of Programs written for Arduino Boards

It is possible to integrate a part into the document, see 29.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document.

Attention: The integration of programs with the help of images is pointless.

Listing 29.1.: The program “Hello World” for an Arduino microcontroller boardis inserted from the file Blink.ino.

```
/**
*
*  @file Blink.ino
*
*  @brief Simple program for testing the configuration
*
*
*  Turns an LED on for one second, then off for one second, repeatedly.
*
*  Most Arduinos have an on-board LED you can control. On the UNO, MEGA and ZERO
*  it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is set to
*  the correct LED pin independent of which board is used.
*  If you want to know what pin the on-board LED is connected to on your Arduino
*  model, check the Technical Specs of your board at:
*
*  https://www.arduino.cc/en/Main/Products
*
*  modified 8 May 2014
*  by Scott Fitzgerald
*  modified 2 Sep 2016
*  by Arturo Guadalupi
*  modified 8 Sep 2016
*  by Colby Newman
*
*  This example code is in the public domain.
*
*  https://www.arduino.cc/en/Tutorial/BuiltInExamples/Blink
*/



/**
*  @brief the setup function runs once when you press reset or power the board
*
*  standard function of Arduino sketches
*
*  Initialization of the pin LED_BUILTIN as output
*
*  @param ---
*
*  @return void
*/
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

/**
*  @brief the loop function runs over and over again forever
*
*  standard function of Arduino sketches
*
*  swichting the led on / off for 1sec.
*
*  @param ---
*
*  @return void
*/

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);   // turn the LED on (HIGH is the voltage level)
  delay(1000);                       // wait for a second
  digitalWrite(LED_BUILTIN, LOW);    // turn the LED off by making the voltage LOW
  delay(1000);                       // wait for a second
}
```

../../Code/Arduino/Blink/Blink.ino

30. First Chapter

...
A Speicherprogrammierbare Steuerung (SPS) is ...

A Computerized Numerical Control (CNC) needs a SPS (PLC) to ...

31. CAGD

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.
The book CAGD by Gerald Farin is a classic on splines. [Far02]

The standard 66025 for programming CNC machines is also a classic; however, it does not deal with splines. [DIN66025-2]

Mr. F. Farouki has dealt with both CNC machine programming and splines. His article[1] on a real-time interpolator also shows this. [FS17]

A new dimension of machine tools have emerged with the invention of 3D printers. [Rus+07]

Another aspect of automation technology is communication. Another milestone has been reached with 5G technology. another milestone has been reached.[2]

[1] co-author is J. Srinathu
[2] Zaf+20.

A. drawings with tikz

The package tikz is a powerful tool for creating graphics. Many introductions exist. Here only the first steps are shown, so that you can easily create flowcharts.

Drawing a line and arrows

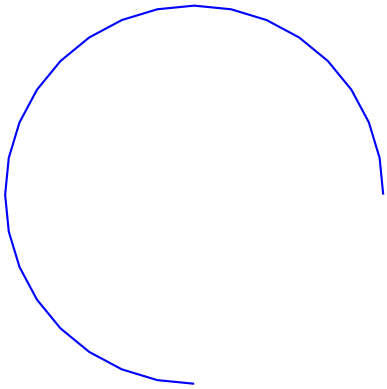
```
\begin{tikzpicture}
    \draw (0,0) -- (1,0);

    \draw[->] (2,0) -- (3,0);

    \draw[<->] (5,0) -- (6,0);
\end{tikzpicture}
```


Drawing a thick blue line

```
\begin{tikzpicture}
    \draw[line width=2pt, blue] (0,0) -- (1,0);

    \draw[line width=2pt, red, dotted] (2,0) -- (3,0);

    \draw[line width=2pt, dashed, green] (4,0) -- (5,0);

    \draw [thick,dash dot] (0,1) -- (5,1);
    \draw [thick,dash pattern={on 7pt off 2pt on 1pt off 3pt}] (0,2) -- (5,2);
\end{tikzpicture}
```

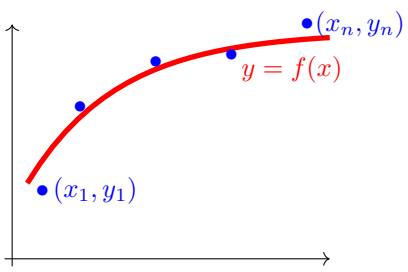
Drawing an arc

```
\begin{tikzpicture}
  \draw [blue,thick,domain=0:270] plot ({5+2.5*cos(\x)}, {1+2.5*sin(\x)});
\end{tikzpicture}
```



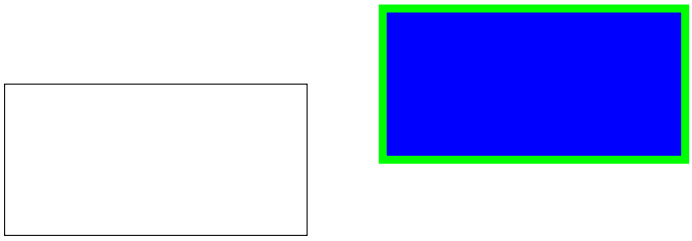
Draw a function

```
\begin{tikzpicture}[
    declare function={%
        F(\x)                    =3-2*pow(2.7979,-0.8*\x);
    }
 ]


    % Zeichnen der Funktion
    \draw[red,line width=2pt,domain=0:4] plot({\x},{F(\x)});

    % Bezeichnung
    \node[red] (O) at(3.5,2.5) {$y=f(x)$};

    % Punkte
    \node[blue] (P1n) at (0.9,0.9) {$(x_1,y_1)$};
    \node[blue] (P1) at (0.2,0.9) {$\bullet$};

   \node[blue] (P2) at (2.7,2.7) {$\bullet$};
   \node[blue] (P3) at (1.7,2.6) {$\bullet$};
   \node[blue] (P4) at (0.7,2) {$\bullet$};

   \node[blue] (P5n) at (4.4,3.1) {$(x_n,y_n)$};
   \node[blue] (P5) at (3.7,3.1) {$\bullet$};

   % Koordinatensystem
   \draw[color=black,->] (-0.2,-0.1) -- (-0.2,3.1);
   \draw[color=black,->] (-0.3,0) -- (4,0);
\end{tikzpicture}
```



Drawing rectangles and moving objects

```
\begin{tikzpicture}
    \draw (0,0) -- (4,0) -- (4,2) -- (0,2) -- cycle;

    \begin{scope}[shift={(5,1)}]
      \draw[green, fill=blue, line width=3pt] (0,0) -- (4,0) -- (4,2) -- (0,2) -- cycle;
    \end{scope}
\end{tikzpicture}
```

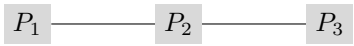


Use of variables

```
\begin{tikzpicture}
\pgfmathsetmacro{\PHI}{-15}
% Now use \PHI anywhere you want -15 to appear,
% can also be used in calculations like 2*\PHI
\def\x{10};

\draw[red] (0,4) -- (1-\PHI*0.5,4);
\draw[green] (0,2) -- (1+1/\x,2);
\draw[blue] (0,0) -- ({1+3*(\x/5+1)},0);
\end{tikzpicture}

\bigskip

%\usetikzlibrary{math} %needed tikz library

\begin{tikzpicture}
  \pgfmathsetmacro{\drehpunktx}{11.63815573}
  %Variables must be declared in a tikzmath environment but
  % can be used outside
  \tikzmath{\x1 = 1; \y1 =1;
  %computations are also possible
  \x2 = \x1 + 1; \y2 =\y1 +3; }
  \draw[->] (\x1, \y1)--(\x2, \y2);
\end{tikzpicture}
```

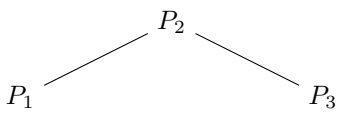
Use of points

```
%\usetikzlibrary{backgrounds}  % is needed

  \begin{tikzpicture}
    \node [fill=gray!30] (P1) at (0,0) { $P_1$ };
    \node [fill=gray!30] (P2) at (2,0) { $P_2$ };
    \node [fill=gray!30] (P3) at (4,0) { $P_3$ };

    \begin{scope}[on background layer]
        \draw (P1) -- (P3);
    \end{scope}
  \end{tikzpicture}
```



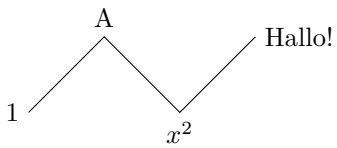
Use of nodes

```
  \begin{tikzpicture}
    \node  (P1) at (0,0){$P_1$};
    \node  (P2) at (2,1){$P_2$};
    \node  (P3) at (4,0){$P_3$};

    \begin{scope}
        \draw (P1) -- (P2) -- (P3);
    \end{scope}
\end{tikzpicture}
```



Use of nodes

```
\begin{tikzpicture}
  \draw (0, 0) node[left]{1}
        -- ++(1, 1) node[above] {A}
        -- ++(1,-1) node[below] {$x^2$}
        -- ++(1, 1) node[right] {Hallo!};
\end{tikzpicture}
```



B. Criteria for a good LaTeX project

A good report is not only characterised by good content, but also fulfils formal aspects. The following list should help to comply with basic rules. Before submitting, all points should be checked and ticked off.

- ☐ Is a suitable directory structure used?
- ☐ Is an appropriate division into files used?
- ☐ Are meaningful directory and file names used?
  - ☐ Are directory and file names such as report or term paper avoided?
  - ☐ Are meaningful names used for images and not „image1“?
  - ☐ Are directory and file names not too long?
  - ☐ Are special characters and spaces avoided?
- ☐ Are commands like \newline and \\ avoided?
- ☐ Are the LaTeXfiles clearly laid out?
  - ☐ Are indentations used in the LaTeXfiles?
  - ☐ Are free lines inserted?
  - ☐ Is the project mentioned in the header of the files?
  - ☐ Does the header of the files mention the main sources?
  - ☐ Is the author mentioned in the header of the files?
- ☐ Are all necessary files given?
- ☐ Are the temporary files deleted?
- ☐ Are graphics created by yourself?
- ☐ Are the sources of the images given?
- ☐ Are citations made with the command \cite?
- ☐ Is a bib file used?
- ☐ Are the entries of the bib-file clearly arranged?
- ☐ Are the entries of the bib-file edited to show them correctly?
- ☐ Are the correct types used for the bib entries?
- ☐ Are meaningful keys used in the bib files?

Unfortunately, the list cannot be complete, but it provides some clues.

C. Model Card

- Model Details - Basic information about the model
  - Person or organization developing model
  - Model date
  - Model version
  - Model type
  - Information about training algorithms, parameters, fairness constraints or other applied approaches, and features
  - Paper or other resource for more information
  - Citation details
  - License
  - Where to send questions or comments about the model
- Intended Use. Use cases that were envisioned during development.
  - Primary intended uses
  - Out-of-scope use cases
- Factors: Factors could include demographic or phenotypic groups, environmental conditions, technical attributes, or others
  - Relevant factors
  - Evaluation factors
- Metrics: Metrics should be chosen to reflect potential real world impacts of the model.
  - Model performance measures
  - Decision thresholds
  - Variation approaches
- Evaluation Data: Details on the dataset(s) used for the quantitative analyses in the card.
  - Datasets
  - Motivation
  - Preprocessing
- Training Data: May not be possible to provide in practice
  - When possible, this section should mirror Evaluation Data. If such detail is not possible, minimal allowable information should be provided here, such as details of the distribution over various factors in the training datasets.
- Quantitative Analyses
  - Unitary results
  - Intersectional results
- Ethical Considerations
- Caveats and Recommendation

Literature

[AB22] A. C. AB. *Axis Communications: License Plate Recognition Cameras*. https://www.axis.com/. 2022.

[AG23] J. AG. *Jenoptik Traffic Solutions: Automatic Number Plate Recognition Systems*. https://www.jenoptik.com/. 2023.

[AQ+20] M. Al-Qizwini et al. "Automatic Number Plate Recognition: A Survey". In: *IEEE Access* (2020).

[Aba+16] M. Abadi et al. "TensorFlow: A System for Large-Scale Machine Learning". In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, pp. 265–283. ISBN: 978-1-931971-33-1. URL: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi.

[BWL20] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao. "YOLOv4: Optimal Speed and Accuracy of Object Detection". In: *arXiv preprint arXiv:2004.10934* (2020).

[Bds] *Bundesdatenschutzgesetz (BDSG) – Federal Data Protection Act, Germany*. https://www.gesetze-im-internet.de/bdsg_2018/. 2021.

[Com22] E. Commission. *European Number Plate Formats and Standards*. Regulatory/Technical Note. 2022.

[Cor22] N. Corporation. *NVIDIA Jetson Nano Developer Kit Datasheet*. 2022.

[DIN66025-2] DIN Deutsches Institut für Normung e.V. *DIN 66025-2:1988-09: Programmaufbau für numerisch gesteuerte Arbeitsmaschinen: Wegbedingungen und Zusatzfunktionen*. Norm. Berlin, Sept. 1988.

[Du+20] S. Du et al. "Automatic Number Plate Recognition: A Review and Outlook". In: *IEEE Transactions on Intelligent Transportation Systems* 21.11 (2020), pp. 4939–4956.

[Eur21] E. Europe. *ANPR in European Intelligent Transport Systems: Practices and Challenges*. Report. 2021.

[FS17] R. Farouki and J. Srinathu. "A real-time CNC interpolator algorithm for trimming and filling planar offset curves". In: *Computer-Aided Design* 86 (Jan. 2017). DOI: 10.1016/j.cad.2017.01.001.

[Far02] G. Farin. *Curves and Surfaces for CAGD*. 5. [pdf]. San Diego, CA: Academic Press, 2002.

[Flu22] Flughafen München GmbH. *Munich Airport: Smart parking and automatic billing systems*. https://www.munich-airport.com/. Accessed: 2025-10-04. 2022.

[Fra23] Fraport AG. *Fraport introduces automated license plate recognition at parking facilities*. https://www.fraport.com/en.html. Accessed: 2025-10-04. 2023.

[GS21] R. Gupta and D. Singh. "Transfer Learning for Efficient Object Detection on Embedded Devices". In: *IEEE Embedded Systems Letters* (2021).

[Gdp] *Regulation (EU) 2016/679 of the European Parliament and of the Council (GDPR)*. https://eur-lex.europa.eu/eli/reg/2016/679/oj. Accessed: 2025-10-04. 2016.

[Goo11] Google, ed. *Coral: Build beneficial and privacy preserving AI*. 14/11/2019. URL: https://coral.withgoogle.com/.

[HDC20] L. Huawei Device Co., ed. *Huawei Kirin 990*. https://consumer.huawei.com/en/campaign/kirin-990-series/Huawei Kirin 990. 2020.

[Hos+21] M. Hossain et al. “Smart Parking Systems Using Computer Vision: A Review”. In: *Sensors* (2021).

[Hua+17] J. Huang et al. “Speed/Accuracy Trade-Offs for Modern Convolutional Object Detectors”. In: *CVPR*. 2017.

[Int19] Intel Corporation, ed. *Intel Neural Compute Stick 2 - Data Sheet*. `../IntelNCS2/ExterneDokumente/Intel/NCS2_Datasheet-English.pdf`pdf. 2019. URL: `https://ark.intel.com/content/www/de/de/ark/products/140109/intel-neural-compute-stick-2.html`.

[JCQ23] G. Jocher, A. Chaurasia, and J. Qiu. “YOLOv8: Next-Generation Object Detection and Image Segmentation Model”. In: *Ultralytics Technical Report* (2023).

[Jai20] JaidedAI. *EasyOCR: Ready-to-Use OCR with 80+ Languages*. GitHub repository. 2020.

[Jak+10] G. Jaklic et al. “On Interpolation by Planar Cubic G^2 Pythagorean-Hodograph Spline Curves”. In: *Mathematics of Computation* (2010). [pdf].

[KSL21] A. Kumar, R. Singh, and J. Li. “Intelligent Transportation Systems: A Review of ANPR Technologies and Applications”. In: *IEEE Transactions on Intelligent Transportation Systems* 22.7 (2021), pp. 4121–4135.

[Kur21] A. Kurniawan. *IoT Projects with NVIDIA Jetson Nano: AI-Enabled Internet of Things Projects for Beginners*. Berkeley, CA: Apress, 2021. ISBN: 978-1-4842-6452-2. DOI: `10.1007/978-1-4842-6452-2_1`. URL: `https://doi.org/10.1007/978-1-4842-6452-2_1`.

[Lar+18] R. Laroca et al. “A Robust Real-Time Automatic License Plate Recognition Based on YOLO Detector”. In: *arXiv preprint arXiv:1807.11081*. 2018.

[MS22] K. Mishra and A. Sharma. “Edge Computing for Smart Surveillance Systems: A YOLO-based Approach”. In: *IEEE Internet of Things Journal* (2022).

[Mit22] S. Mittal. *Edge AI: Concepts and Applications*. Springer, 2022.

[Muf+21] N. Mufti et al. “Automatic Number Plate Recognition: A Detailed Survey”. In: *Sensors* (2021).

[NVI19] NVIDIA Corporation, ed. *Getting Started With Jetson Nano Developer Kit*. 2019. URL: `https://developer.nvidia.com/embedded/learn/get-started-jetson-nano-devkit`.

[NVI20] NVIDIA Corporation, ed. *Jetson Community Projects*. 2020. URL: `https://developer.nvidia.com/embedded/community/jetson-projects`.

[NVI21] NVIDIA Corporation. *NVIDIA Jetson Platform for Edge AI: Technical Overview*. White Paper. 2021.

[NVI22] NVIDIA Corporation. *NVIDIA Jetson Nano Developer Kit: Technical Overview*. `https://developer.nvidia.com/embedded/jetson-nano-developer-kit`. 2022.

[NVI23] NVIDIA. *NVIDIA Jetson Platform for Edge AI*. Whitepaper/Developer Docs. 2023.

[Net+24] P. Netinant et al. “Evaluating Factors Shaping Real-Time Internet-of-Things License Plate Recognition Systems”. In: *MDPI Sensors* (2024).

[Ope19] OpenALPR. *OpenALPR Benchmark Dataset*. Dataset. 2019.

[Pet23] Peter Park System GmbH. *Peter Park – Smart parking management using AI-based license plate recognition*. `https://www.peter-park.de/`. Accessed: 2025-10-04. 2023.

[Pla+24] N. Plavac et al. “Performance of Automatic License Plate Recognition Systems on Distorted Images”. In: *Journal of Imaging Science and Technology* (2024).

[Rao+24] P. Rao et al. “Evaluation of YOLOv8 for Real-Time Object Detection Applications”. In: *IEEE Access* (2024).

[Red+16a] J. Redmon et al. “You Only Look Once: Unified, Real-Time Object Detection”. In: *CVPR*. 2016.

[Red+16b] J. Redmon et al. “You Only Look Once: Unified, Real-Time Object Detection”. In: *CVPR*. 2016.

[Ren+15] S. Ren et al. “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks”. In: *NeurIPS*. 2015.

[Rus+07] D. Russell et al. *Apparatus and Methods for 3D Printing*. United States Patent, Patent N0.: US 7,291,002 B2. 2007.

[SD20] W. Shi and S. Dustdar. “Privacy-Preserving Analytics on the Edge: A Survey”. In: *IEEE Internet Computing* (2020).

[Sam20] Samsung, ed. *Samsung Exynos-980*. `https://www.samsung.com/semiconductor/minisite/exynos/products/mobileprocessor/exynos-980/`Samsung Exynos-980. 2020.

[Smi07] R. Smith. “An Overview of the Tesseract OCR Engine”. In: *Proc. Ninth Int. Conf. on Document Analysis and Recognition (ICDAR)* (2007), pp. 629–633.

[Stu23] S. Studio. *Seeed Studio reComputer J1010 Carrier Board Datasheet*. 2023.

[Sul+23] F. Sultan et al. “Towards Automatic License Plate Recognition in Varied Lighting Conditions”. In: *Applied Sciences* (2023).

[TPL20] M. Tan, R. Pang, and Q. V. Le. “EfficientDet: Scalable and Efficient Object Detection”. In: *CVPR* (2020).

[Tan+22] J. Tang et al. “Automatic Number Plate Recognition in Smart Cities: A Survey”. In: *Journal of Smart City Applications* (2022).

[Tav+24] R. A. Tavares et al. “Comparison of Image Preprocessing Techniques for OCR under Varied Lighting Conditions”. In: *ArXiv / Preprint* (2024).

[Tec21] Technische Hochschule Ingolstadt. *5GoIng: Test field for automated and connected driving at THI Ingolstadt*. `https://www.thi.de/go/5going`. Accessed: 2025-10-04. 2021.

[Ult23] Ultralytics. *YOLOv8 Documentation*. `https://docs.ultralytics.com/`. 2023.

[Vin18] J. Vincent. “Google unveils tiny new AI chips for On-Device Machine Learning”. In: *The Verge* (2018). URL: `https://www.theverge.com/2018/7/26/17616140/google-edge-tpu-on-device-ai-machine-learning-devkit`.

[WCL23] X. Wang, Y. Chen, and Z. Li. “A Comparative Study of YOLOv5 and YOLOv8 for Real-Time Object Detection”. In: *IEEE Access* (2023).

[Wat17a] Waterloo Maple Inc. 2017. *Description*. letzter Zugriff 14.09.2017. 2017. URL: `https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/description`.

[Wat17b] Waterloo Maple Inc. 2017. *Export*. letzter Zugriff 14.09.2017. 2017. URL: `https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/export`.

[Wat17c] Waterloo Maple Inc. 2017. *Module*. letzter Zugriff 14.09.2017. 2017. URL: `https://de.maplesoft.com/support/help/maple/view.aspx?path=module`.

[Wat17d] Waterloo Maple Inc. 2017. *ModuleLoad*. letzter Zugriff 14.09.2017. 2017. URL: `https://de.maplesoft.com/support/help/Maple/view.aspx?path=ModuleLoad`.

[Wat17e] Waterloo Maple Inc. 2017. *Option*. letzter Zugriff 14.09.2017. 2017. URL: `https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/option`.

[Wat17f] Waterloo Maple Inc. 2017. *Procedures*. letzter Zugriff 14.09.2017. 2017. URL: `https://de.maplesoft.com/support/help/Maple/view.aspx?path=procedure`.

[Xu+18] Z. Xu et al. “Towards End-to-End License Plate Detection and Recognition: CCPD Dataset”. In: *ECCV Workshops*. 2018.

[ZC20] Q. Zhou and H. Chen. “Real-Time Object Detection and Recognition on Edge Devices”. In: *IEEE Access* 8 (2020), pp. 123456–123467.

[ZSW23] L. Zhao, X. Sun, and P. Wang. “Edge AI: A Survey on Technologies, Architectures, and Applications”. In: *IEEE Internet of Things Journal* (2023).

[ZXJ21] L. Zhang, W. Xu, and Q. Jiang. “Real-Time Vehicle Detection and License Plate Recognition Based on YOLO and OCR”. In: *Sensors* 21.4 (2021), p. 1232.

[Zaf+20] A. Zafeiropoulos et al. “Benchmarking and Profiling 5G Verticals’ Applications: An Industrial IoT Use Case”. In: *IEEE Conference on Network Softwarization, NetSoft 2020*. 2020. DOI: 10.1109/NetSoft48620.2020.9165393.

Index

3D printer, 145

Programmable Logic Controller, 143
File
    Blink.ino, 142
    Blink.py, 140
    PackageExample.py, 111
    requirements.txt, 109

CAGD, 145
    Splines, 145
CNN, 13, 143
Computerized Numerical Control
    see CNN, 13, 143

Example, 111
    Description, 111
    Installation, 111
    Introduction, 111

PLC, *see* Programmable Logic Controller

Speicherprogrammierbare Steuerung
    see SPS, 13, 143
SPS, 13, 143